

How To GreasyPidgin

Contents

1. Was ist GreasyPidgin?	1
2. Installation	3
2.1 Python installieren	3
2.2 Optional: Installation von miniconda	3
2.3 Installation von GreasyPidgin	3
3. Pitch	6
3.1 Aufbau	6
3.2 Umrechnungen	6
4. Envelope	9
4.1 Konstruktoren	9
4.2 Werte ausgeben	9
4.3 Envelope anzeigen	10
5. Grid	12
5.1 Ein Grid erzeugen	12
5.2 Quantisieren	14
5.3 Sieben	14
5.4 Sampling	17
6. TimeGrid	19
6.1 Vererbte Grid-Konstruktoren	19
6.2 Mikrotime-basierte Konstruktoren	20
6.3 Midi-Export	21
7. PitchGrid	23
7.1 Stimmungssysteme	23
7.2 Skalen	24
7.3 Konstruktoren	25
7.4 Export	26
8. DynamicGrid	29
9. TemporalObject	31
9.1 Was ist ein TemporalObject?	31
9.2 Aufbau	31
9.3 Ein TemporalObject erzeugen	31
9.4 Ein TemporalObject exportieren	32
9.5 MIDI-Export im Detail	32
9.6 Baumstruktur	34
10. Phonon	36
10.1 Was ist ein Phonon?	36
10.2 Ein Phonon erzeugen	36
10.3 Tonhöhenstruktur	37
10.4 Glissando	39
10.5 unfold() — Tonhöhen auflösen	39
10.6 Export	39
11. Gesture	42
11.1 Was ist eine Gesture?	42
11.2 Eine Gesture erzeugen	42

11.3	Export	44
12.	Zellulärer Automat à la Wolfram	48
12.1	Was ist ein Zellulärer Automat?	48
12.2	Gesamtüberblick	48
12.3	Schritt 1 – Importe	48
12.4	Schritt 2 – Grundlegende Parameter	49
12.5	Schritt 3 – Das Tonhöhengitter aufbauen	49
12.6	Schritt 4 – Den Zellulären Automaten einrichten	50
12.7	Schritt 5 – Den ZA auf Tonhöhen abbilden	51
12.8	Schritt 6 – Den ZA auf Rhythmen abbilden	51
12.9	Schritt 7 – Die Geste aufbauen	52
12.10	Experimente	52
12.11	Ausblick: Die Stärke algorithmischer Generierung	52
13.	Markov-Melodie	56
13.1	Was ist eine Markov-Kette?	56
13.2	Gesamtüberblick	56
13.3	Schritt 1 – Importe	56
13.4	Schritt 2 – Übergangsregeln definieren	56
13.5	Schritt 3 – Die Markov-Kette aufbauen	57
13.6	Schritt 4 – Sequenz generieren	57
13.7	Schritt 5 – Geste aufbauen und exportieren	57
13.8	Experimente	57
14.	Markov-Rhythmus	60
14.1	Rhythmus als Markov-Kette	60
14.2	Gesamtüberblick	60
14.3	Schritt 1 – Importe	60
14.4	Schritt 2 – Rhythmische Übergangsregeln definieren	60
14.5	Schritt 3 – Sequenz generieren	61
14.6	Schritt 4 – Skalierung auf Taktschläge	61
14.7	Schritt 5 – Tonhöhen zufällig auswählen	61
14.8	Schritt 6 – Geste aufbauen und exportieren	62
14.9	Experimente	62
15.	Markov-Intervalle	64
15.1	Intervalle statt Tonhöhen	64
15.2	Gesamtüberblick	64
15.3	Schritt 1 – Importe	64
15.4	Schritt 2 – Intervall-Regeln definieren	64
15.5	Schritt 3 – Intervallsequenz generieren	65
15.6	Schritt 4 – Intervalle in Tonhöhen umrechnen	65
15.7	Schritt 5 – Geste aufbauen und exportieren	66
15.8	Experimente	66
16.	Markov-Melodie mit Markov-Rhythmus	68
16.1	Zwei Ketten, eine Musik	68
16.2	Gesamtüberblick	68
16.3	Schritt 1 – Importe	68
16.4	Schritt 2 – Intervall-Kette (Tonhöhen)	68
16.5	Schritt 3 – Rhythmus-Kette	69

16.6	Schritt 4 – Beide Ketten kombinieren	69
16.7	Experimente	70

1. Was ist GreasyPidgin?

GreasyPidgin ist algorithmische Kompositionsumgebung, die versucht möglichst keine Vorgaben zu präsentieren, sondern mittels modularem Aufbau verschiedene Arten einer kreativen Anwendung zu ermöglichen. Nichtsdestotrotz gibt es eine kleine Anzahl von Festlegungen. GreasyPidgin basiert in erster Linie auf drei fundamentalen Systemen:

TemporalObject

Grid

Envelope

Hinzu kommen Erweiterungen, die sich an Konventionen traditioneller Komposition orientieren, wie

Ensemble

Player

Instrument

Pitch

GreasyPidgin beinhaltet außerdem eine Reihe algorithmischer Verfahren, die es erlauben Strukturen von Daten¹ zu erzeugen:

Lindenmayer-Systeme

Markov-Ketten

Zelluläre Automaten

Sieb- und Mengen- Operationen

Sowie dezidiert musikhistorisch algorithmische Verfahren:

Isorhythmie

Serielle Verfahren

¹...und somit auch klangliche bzw. musikalische Parameter

2. Installation

GreasyPidgin ist eine Umgebung, die vor allem in Python geschrieben ist, daher ist der erste Schritt sicherzustellen, dass Python installiert ist.

2.1 Python installieren

Unter <https://www.python.org/downloads> lässt sich die aktuelle Version von Python

2.2 Optional: Installation von miniconda

Python hat eine Tendenz, dass verschiedene Projekte verschiedene Versionen von Libraries benötigen, welche wiederum verschiedene Versionen von Python benötigen. Um sicherzustellen, dass bei der Arbeit mit GreasyPidgin keine Konflikte auftreten, empfiehlt es sich ein virtuelles Environment anzulegen, innerhalb dessen ausschließlich GreasyPidgin verwendet wird. Eine Möglichkeit dafür bietet Conda, zum Beispiel in der Variante miniconda

Unter <https://www.anaconda.com/docs/getting-started/miniconda/install> lässt sich die aktuelle Version von miniconda finden sowie Hinweise zur Installation. Um ein miniconda Environment für GreasyPidgin anzulegen, öffne ein Terminal und führe folgenden Code aus:

```
conda create --name greasypidgin python=3.14
```

um es zu aktivieren:

```
conda activate greasypidgin
```

2.3 Installation von GreasyPidgin

Lade das Repository von <https://gitea.com/sem-16161d/GreasyPidgin> wie folgt herunter. Klicke auf **Code** -> **Download ZIP** und entpacke die Datei in dem Verzeichnis, wo es landen soll. Öffne ein Terminal, und wechsele auf den Ordner, den du angelegt hast via `cd - change directory` – wie folgt:

```
cd path/to/directory/of/GreasyPidgin
```

Um GreasyPidgin zu installieren führe eine lokale Installation via `pip` aus. Stelle zuvor sicher, dass das GreasyPidgin Environment aktiv ist!

```
pip install -e .
```

Um sicherzustellen, dass GreasyPidgin installiert in dem Environment installiert ist, starte Python via

```
python3
```

Oder

```
python
```

Und führe dann in der Python-shell folgenden Code aus:

```
from GreasyPidgin.GreasyPidgin import GreasyPidgin
print(GreasyPidgin())
```

Im Terminalfester solltest du dann folgenden Printout erhalten:

```
#####
```

```
  .-.-.
  ( @° \
  < . . . \
  ) . . . \
  /-.-.-.- \
  | GREASY  ( ) \-.-.
  \ PIDGIN  ( ) ( ) \);) \-.-, _
```

```
\,      _:(0) 0)0_) ;););)-->>
" ,    / , - - - 
 \, / 
 ,_ // 
 ,_- '- - ,
```

```
name      : GreasyPidgin
id        : 0
start     : 0.000
dur       : 0.000
ensemble  : Ensemble(players=0, names=[],
tmp0bjs   : 0
```


3. Pitch

Eine Python-Adaption von Michael Edwards' `Pitch.lsp` aus seiner Kompositionsumgebung `slippery chicken`². `Pitch.py` enthält dabei keine eigenständige Klasse, sondern ist lediglich eine Sammlung von Funktionen. Das System basiert auf der *Scientific Pitch Notation (SPN)*³.

3.1 Aufbau

3.1.1 Oktavlagen

In diesem System ist der tiefste Ton als $c_0 = 16.3516 \text{ Hz}$ definiert, bei einem Kammerton von $a_4 = 440 \text{ Hz}$. Die Basis bilden die englischen Notennamen: A, B, C, D, E, F, G gefolgt von der Oktavlage von 0-9.

SPN	DE	MIDI	Hz ($a=440$)	Hz ($a = 442$)
c0	Subkontra C	12	16.35	16.42
c1	Kontra C	24	32.70	32.85
c2	großes C	36	65.4	65.70
c3	kleines c	48	130.81	131.4
c4	c'	60	261.626	262.81
c5	c''	72	523.25	525.63
c6	c'''	84	1046.5	1051.26
c7	c''''	96	2093	2102.52
c8	c'''''	108	4186	4205

3.1.2 Versetzungszeichen

Vorzeichen / Versetzungszeichen werden mittels Suffix indiziert: f für *flat* und s für *sharp*.

SPN	DE	MIDI	Hz ($a=440$)	Hz ($a = 442$)
af4	as'	68	415.3	417.19
cs3	kleines cis	49	138.59	139.22

Das gleiche gilt für Mikrotonalität⁴:

Suffix	EN	DE	CENT
qf	quarter tone flat	Viertelton tief	-50
qs	quarter tone sharp	Viertelton hoch	+50
sf	sixth tone flat	Sechstelton tief	-33
ss	sixth tone sharp	Sechstelton hoch	+33
tf	twelfth tone flat	Zwölftelton tief	-17
ts	twelfth tone sharp	Zwölftelton hoch	+17

3.2 Umrechnungen

Um zwischen den verschiedenen Tonhöhendarstellungen zu wechseln sind die Funktionen nach ihrer Entsprechungen in PureData und MAX/MSP benannt. Die Umrechnung Frequenz zu MIDI (*fom*) folgt der Formel⁵:

²<https://michael-edwards.org/sc/>

³wiki: [Scientific pitch notation](#)

⁴Edwards' Original nimmt EDO-72 als Standard an. Auch für den Gebrauch in einem Just-Intonation-System lassen sich die ersten 16 Partialtöne hinreichend genau mit diesem System beschreiben.

⁵Eine gute Herleitung und Zusammenfassung findet sich hier: <https://newt.phys.unsw.edu.au/jw/notes.html>

$$f(a_{ref}, MIDI) = a_{ref} * 2^{(MIDI-69)/12} \quad (3.1)$$

Und umgekehrt MIDI zu Frequenz (*mtof*):

$$MIDI(a_{ref}, f) = 69 + 12 * \log_2(f/a_{ref}) \quad (3.2)$$

Der default-Wert für den Referenz-Kammerton a_{ref} ist 440 Hz, lässt sich aber via *referenceA4* auf eine beliebige Frequenz ändern. *spntom* und *spntof* sind die Äquivalente um *SPN* in MIDI und Frequenz umzurechnen. Eine direkte Umwandlung von MIDI oder *Frequenz* in *SPN* ist zur Zeit noch nicht implementiert, da dieses die Bestimmung von Stimmungssystemen voraussetzt. Eine solche Umwandlung muss über *PitchGrid* erfolgen.

```
from GreasyPidgin.Pitch import mtof, ftom, spntom, spntof
pitch = 'c4'
midi = spntom(pitch)
freqDirect = spntof(pitch, referenceA4=440)
freqViaMtof = mtof(midi)
back2Midi = ftom(freqDirect)

print(
    " pitch          : ", pitch, '\n',
    "midi           : ", midi, '\n',
    "freq           : ", freqDirect, '\n',
    "freq(from Midi) : ", freqViaMtof, '\n',
    "freq to midi    : ", back2Midi, '\n',
)
```

OUTPUT

```
#####
#> pitch          : c4
#> midi           : 60.0
#> freq           : 261.6255653005986
#> freq(from Midi) : 261.6255653005986
#> freq to midi    : 60.0
```


4. Envelope

Jegliche Art von zweidimensionalen Abfolgen, Kurven und Hüllkurven wird in GreasyPidgin als Envelope dargestellt. Ein Envelope-Objekt kann entweder aus einer Reihe von Werten oder Wertepaaren – sogenannten *breakpoint pairs* – erzeugt werden. Während der Konstruktion werden die Werte innerhalb des Objekts normalisiert. Intern wird eine Envelope als Kurve in einem XY-Koordinatensystem gesehen.

4.1 Konstruktoren

4.1.1 Einzelwerte

Wird Envelope mit einem flachen list- oder tuple-Objekt initialisiert, werden die enthaltenen Werte in *breakpoint pairs* umgewandelt. Jedes Paar hat den gleichen x-Abstand zum benachbarten Paar.

```
from GreasyPidgin.Envelope import Envelope
```

```
env = Envelope([0,2,1,3,2,5,2,3,0,1,0])
print(env)
```

```
##### OUTPUT
```

```
#####
```

```
# Envelope
#   pointsNorm=[(0.0, 0.0), (0.1, 0.4), (0.2, 0.2), (0.3, 0.6), (0.4, 0.4),
#               (0.5, 1.0), (0.6, 0.4), (0.7, 0.6), (0.8, 0.0), (0.9, 0.2),
#               (1.0, 0.0)]
#   x_range=(0.0, 10.0), y_range=(0.0, 5.0)
#   coefficients={}
```

4.1.2 Wertepaare

Wird Envelope mit einem verschachtelten list oder tuple initialisiert, müssen die einzelnen Elemente **immer** x- und y-Wertpaare sein. In der Standardeinstellung werden die Elemente anhand ihres x-Wertes sortiert.

```
from GreasyPidgin.Envelope import Envelope
env = Envelope([(0,5),(1,1),(5,1),(4,2)])
print(env)
```

4.1.3 Polynom-Berechnung

Eine Möglichkeit Daten zu komprimieren ist die Berechnung von polynomischen Funktionen verschiedener Ordnung.⁶ Dies erlaubt zum Beispiel alternative Fortführungen der Kurve außerhalb des definierten Wertebereiches (siehe: *Werte ausgeben*)

4.2 Werte ausgeben

Mit der Methode `.getValue()` können Werte innerhalb der Kurve ausgegeben werden. Dabei gibt es sowohl die Möglichkeit x- als auch y-Werte normalisiert zu verwenden.

```
from GreasyPidgin.Envelope import Envelope
```

```
env = Envelope([0,1,2,3,2,1,0])
# get the y-value for the position 0.5 of 6
print("0.5 of 6: ",env.getValue(xPos=0.5, normalizedX=False))
# get the y-value for the position 0.5 of 1 aka 3 of 6
print("0.5 of 1: ",env.getValue(xPos=0.5, normalizedX=True))
# get the normalized y-value at x = 0.5 of 6
print("y-norm: ",env.getValue(xPos=0.5, normalizedY = True))
# get the normalized y-value at normalized x
print("y-norm @ x-norm: ",env.getValue(xPos=0.5, normalizedX=True, normalizedY
= True))
```

⁶Keine vertieften Mathekenntnisse nötig! Kurzgesagt, umso höher die Ordnung, desto komplexere Auf- und-Ab sind darstellbar. Generell gilt: Es kann nur sichergestellt werden, dass alle Punkte auf der Kurve liegen, wenn die Ordnung der Kurve der Anzahl der Punkte entspricht.

```
##### OUTPUT
#####
# 0.5 of 6: 0.5
# 0.5 of 1: 3.0
# y-norm: 0.16666666666666666
# y-norm @ x-norm: 1.0
```

Wird ein Wert außerhalb des Wertebereichs abgefragt, wird entweder per Standard interpoliert, alternativ kann der letzte gültige Wert ausgegeben werden.

```
from GreasyPidgin.Envelope import Envelope

env = Envelope([0,1,2,3,2,1,0])
print(env.getValue(11))
print(env.getValue(11, interpolateOutsideOfRange=False))
```

```
##### OUTPUT
#####
# -5.0
# 0.0
```

Werden bei der Konstruktion des Objekts Polynome berechnet, können alternativ auch diese innerhalb und außerhalb des Wertebereiches abgerufen werden.

```
from GreasyPidgin.Envelope import Envelope

env = Envelope([0,1,2,3,1,2,4], polyFit=True)
print("just x: ",env.getValue(0.5, normalizedX=True))
print("1st order: ",env.getValue(0.5, normalizedX=True,polyDegree=1))
print("2nd order: ",env.getValue(0.5, normalizedX=True,polyDegree=2))
print("3rd order: ",env.getValue(0.5, normalizedX=True,polyDegree=3))
print("4th order: ",env.getValue(0.5, normalizedX=True,polyDegree=4))
print("5th order: ",env.getValue(0.5, normalizedX=True,polyDegree=5))
```

```
##### OUTPUT
#####
# just x: 3.0
# 1st order: 1.8571428571428568
# 2nd order: 1.9047619047619047
# 3rd order: 1.904761904761903
# 4th order: 2.3722943722943786
# 5th order: 2.372294372294444
```

4.3 Envelope anzeigen

Mittels `.display()` kann die Envelope unter Zuhilfenahme von `matplotlib` dargestellt werden. Wird `show_polynomials=True` gesetzt, können außerdem die Polynome angezeigt werden.

```
from GreasyPidgin.Envelope import Envelope

env = Envelope([0,1.5,1,3,1,2,4], polyFit=True)
env.display(show_polynomials=True)
```


5. Grid

Ein Grid (de: *Raster*) ist ein set, das zur Quantisierung von Daten genutzt werden kann. Die grundlegenden Methoden um ein Grid zu erzeugen sind Übersetzungen der Array-Konstrukturen aus SuperCollider⁷.

5.1 Ein Grid erzeugen

5.1.1 Direktes Initialisieren

Ein Grid kann im einfachsten Fall direkt aus einem set, tuple oder list erzeugt werden. Dabei ist allerdings zu beachten, dass Grid ein set ist und daher jeder Wert einzigartig sein muss und ein list-Objekt nicht verschachtelt sein darf.

```
from GreasyPidgin.Grid import Grid

print(Grid([1, 3, 5, 7]))
print(Grid({2,4,6,8}))
print(Grid((1,2,3)))

##### OUTPUT
#####
#> Grid(size=4, range=(1, 7), preview=[1, 3, 5, 7])
#> Grid(size=4, range=(2, 8), preview=[2, 4, 6, 8])
#> Grid(size=3, range=(1, 3), preview=[1, 2, 3])
```

5.1.2 Die arithmetische Reihe (1,2,3,4,...)

Für die meisten Anwendungsfälle ist eine einfache arithmetische Reihe, die passende Art und Weise ein Grid zu erzeugen. Via `.series(start, step, size)` wird ein Grid erzeugt, das vom Startwert start ausgehend size - 1 mal den Wert step aufsummiert.

```
from GreasyPidgin.Grid import Grid

print(Grid.series(0, 1, 10))
print(Grid.series(0, 0.1, 5))

##### OUTPUT
#####
#> Grid(size=10, range=(0, 9), preview=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
#> Grid(size=5, range=(0.0, 0.4), preview=[0.0, 0.1, 0.2, 0.3, 0.4])
```

5.1.3 Die geometrische Reihe (1, 2, 4, 8,...)

In einer geometrischen Reihe haben zwei benachbarte Reihenglieder stets das gleiche Verhältnis. Via `.geom(start, ration, size, *, decimals = 100)` wird die Reihe konstruiert, indem vom Startwert start ausgehend der letzte Wert der Reihe iterativ mit der ratio multipliziert wird.

```
from GreasyPidgin.Grid import Grid

print(Grid.geom(1, 2, 8))

##### OUTPUT
#####
#> Grid(size=8, range=(1, 128), preview=[1, 2, 4, 8, 16, 32, 64, 128])
```

Diese erlaubt es zum Beispiel die Frequenzen für *equal distance octave (EDO)*-Stimmungen zu berechnen. Mittels des Methodenslots `decimals` lassen sich zusätzlich die Nachkommastellen durch runden reduzieren.

```
from GreasyPidgin.Grid import Grid

print(Grid.geom(442, 2**((1/12)), 13, 2))
```

⁷Siehe <https://docs.supercollider.online/Classes/Array.html>


```
##### OUTPUT
#####
#> Grid(size=13, range=(442.0, 884.0), preview=[442.0, 468.28, 496.13, 525.63,
556.89, 590.0, 625.08, 662.25, 701.63, 743.35, 787.55, 834.38, 884.0])
```

5.1.4 Initialisieren durch lineare Interpolation

Komplementär zur arithmetischen Reihe, bei der die Schrittgröße aber kein Endwert festgelegt ist, erzeugt `.linearInterpolation(start, end, size, *, decimals = 100, includeEnd = True)` ein Grid, indem die Schrittgröße aus `start`, `end` und `size` bestimmt wird. Mittels des Methodenslots `decimals` lassen sich zusätzlich die Nachkommastellen durch runden reduzieren, mittels `includeEnd` lässt sich wählen, ob der Endwert Teil der Reihe ist oder nicht.

```
from GreasyPidgin.Grid import Grid

print(Grid.linearInterpolation(0, 10, 5))
print(Grid.linearInterpolation(0, 10, 5, includeEnd= False))
print(Grid.linearInterpolation(0, 2.111, 5))
print(Grid.linearInterpolation(0, 2.111, 5, decimals=1))
```

```
##### OUTPUT
#####
#> Grid(size=5, range=(0.0, 10.0), preview=[0.0, 2.5, 5.0, 7.5, 10.0])
#> Grid(size=5, range=(0.0, 8.0), preview=[0.0, 2.0, 4.0, 6.0, 8.0])
#> Grid(size=5, range=(0.0, 2.111), preview=[0.0, 0.52775, 1.0555, 1.58325,
2.111])
#> Grid(size=5, range=(0.0, 2.1), preview=[0.0, 0.5, 1.1, 1.6, 2.1])
```

5.1.5 Fibonacci-artiges Grid

`.fib(size, a, b)` erlaubt es ein Fibonacci-artiges Grid zu erstellen, bei dem der nächste Wert immer die Summe beider vorherigen Werte ist. Initialisiert man mit Wertepaaren aus der Fibonacci-Reihe – z.B.: `a= 1, b = 2` – erhält man die Fortsetzung innerhalb der Fibonacci-Reihe⁸.

```
from GreasyPidgin.Grid import Grid

print(Grid.fib(10,1,2))

##### OUTPUT
#####
#> Grid(size=10, range=(1, 89), preview=[1, 2, 3, 5, 8, 13, 21, 34, 55, 89])
```

`.fib` ist allerdings nicht auf `int` begrenzt sondern erlaubt auch `float`-Werte.

```
from math import e, pi
from GreasyPidgin.Grid import Grid

print(Grid.fib(5,e, pi))

##### OUTPUT
#####
#> Grid(size=5, range=(2.718281828459045, 14.861341617687469),
preview=[2.718281828459045, 3.141592653589793, 5.859874482048838,
9.00146713563863, 14.861341617687469])
```

5.1.6 Initialisieren mit Zufallswerten

Man mag sich fragen, warum man es notwendig sein kann Daten auf Zufallswerte zu quantisieren. Aber Grid erlaubt eine Initialisierung mit Zufallswerten, sowohl `int` als auch `float` basiert. Sollen `int`-Werte erzeugt werden, muss dieses explizit über das Argument `integer = True` ausgedrückt werden.

⁸**Achtung!** Auch hier werden doppelte Werte gelöscht! Die tatsächliche Fibonacci-Reihe [0,1,1,2,3,5,8,...] lässt sich nicht als set darstellen!

```

from GreasyPidgin.Grid import Grid

print(Grid.rand(5,0,12.77))
print(Grid.rand(5,0,12.77,integer=True))
print(Grid.rand(5,0,13))
print(Grid.rand(5,0,13,integer=True))

##### OUTPUT
#####
### als Beispiel, duh!
#> Grid(size=5, range=(0.2856196224877912, 7.789117475083042),
preview=[0.2856196224877912, 4.079976016998381, 5.363352927568584,
6.151736837079151, 7.789117475083042])
#> Grid(size=5, range=(2, 11), preview=[2, 8, 9, 10, 11])
#> Grid(size=5, range=(0.44959703668048456, 7.894089566506886),
preview=[0.44959703668048456, 3.3165137492950127, 3.822080408499572,
7.612798394949432, 7.894089566506886])
#> Grid(size=5, range=(2, 11), preview=[2, 3, 5, 9, 11])

```

5.1.7 Initialisieren über eine Funktion

Grid.fill(size, func_or_value) erlaubt es ein Grid mit einer Konstanten⁹ oder mittels einer Funktion zu füllen. Die Funktion kann dabei als anonyme Funktion definiert werden¹⁰. Dies ist vor allem eine Legacy-Option, die in der Praxis eher selten zur Anwendung kommt, und durch das direkte Initialisieren durch den Output einer Funktion in Form von set, tuple oder list abgelöst wird.

```

from GreasyPidgin.Grid import Grid

print(Grid.fill(10, lambda i: 2**i))

##### OUTPUT
#####
#> Grid(size=10, range=(1, 512), preview=[1, 2, 4, 8, 16, 32, 64, 128, 256,
512])

```

5.2 Quantisieren

Wie eingangs erwähnt dient Grid vor allem dazu Daten zu quantisieren. Mittels der Methode .quantise(value) ist dieses möglich. Es können nur int und float quantisiert werden. Sollen ein iterable wie set, list oder tuple quantisiert werden, muss sichergestellt sein, dass diese nur Zahlen enthalten.

```

from GreasyPidgin.Grid import Grid

g = Grid.series(0,0.5, 10)
print(g.quantise(1.23456789))
print(g.quantise([1.2345, 2.3456]))
print(g.quantise((1.2345, 2.3456)))
print(g.quantise({1.2345, 2.3456}))

##### OUTPUT
#####
#> 1.0
#> [1.0, 2.5]
#> (1.0, 2.5)
#> {1.0, 2.5}

```

5.3 Sieben

Eine weitere Funktion von Grid ist die Möglichkeit ein Sieb auf sich selbst anzuwenden¹¹. Die Siebtheorie selbst stammt aus der analytischen Zahlentheorie und bezeichnet eine

⁹**Achtung!** Es wird ein grid mit der Länge 1 erzeugt!

¹⁰Siehe https://www.w3schools.com/python/python_lambda.asp

¹¹Für einen Deepdive des Ursprungs dieser Idee siehe: <https://www.iannis-xenakis.org/en/sieve-theory/>

Reihe von Techniken, mittels derer eine Menge gefiltert wird, so dass am Ende nur die Zahlen enthalten sind, die nicht durch das Sieb gefallen sind.

5.3.1 Hintergrund: Das Sieb des Eratosthenes

Das älteste dokumentierte Sieb, das auch als solches bezeichnet wurde ist das Sieb des Eratosthenes aus dem 3. Jhd vor Christus. Dieses Sieb dient dazu Primzahlen aus einer beliebigen Menge natürlicher Zahlen herauszufiltern. Im ersten Schritt werden dafür alle Zahlen geordnet auf einem Zahlenstrang notiert. In Python entspricht das einer Liste, die mittels `[i+2 for i in range(100-2)]` erzeugt wird. Ausgehend von der 2 als kleinste Primzahl werden via Modulo alle ganzzahligen Vielfachen von 2 aus der Liste entfernt und die zwei als letzte Primzahl an die Liste `primes` gehängt. Die nächste größere Zahl 3 *muss* nun wieder auch eine Primzahl sein.

```
allIntegers = [i+2 for i in range(100-2)]
lenTS = len(allIntegers)
primes = []
```

```
while lenTS > 0:
    newPrime = allIntegers[0]
    primes.append(newPrime)
    for i in allIntegers:
        if i%newPrime == 0:
            allIntegers.remove(i)
    lenTS = len(allIntegers)

print(primes)
```

OUTPUT

#####

```
#> [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67,
71, 73, 79, 83, 89, 97]
```

Diese einfachste Form des Siebs basiert nur auf einem Parameter, der Schrittgröße, in Form der aufsteigenden Primzahlen.

5.3.2 Xenakis

Iannis Xenakis entwickelte seine Form der Siebtheorie während seiner Zeit in Westdeutschland um 1963. Im Gegensatz zu Eratosthenes, ging es Xenakis nicht darum, Primzahlen zu bestimmen, sondern vor allem eine sortierte Menge (*well sorted set*) so zu filtern und mit anderen Filtrierungen der gleichen Menge so zu kombinieren, dass eine neuartige mathematisch fundierte harmonische Sprache entsteht, die sich an keiner bisher etablierten Harmonik – weder tonaler, noch serieller – orientiert. Neu in Xenakis' Denken ist dabei die Einführung eines Offsets, ab dem die Menge gefiltert wird.

```
from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =0)
g.sieve(stepSize = 3, offsetIndex =1)
for s in g.sieved:
    print(g.sieved[s]['sievedGrid'])
```

OUTPUT

#####

```
#> Grid(size=7, range=(0, 18), preview=[0, 3, 6, 9, 12, 15, 18])
#> Grid(size=7, range=(1, 19), preview=[1, 4, 7, 10, 13, 16, 19])
```

5.3.3 Erweiteretes Sieben

In der Implementation von `Grid.sieve()` gibt es nun die Möglichkeit die traditionellen Siebverfahren als auch Erweiterungen dieser zu nutzen. Mittels `stepSize` und `offsetIndex` lassen sich alle Siebe nach Xenakis anwenden. Wird ein Grid gesiebt, wird das Ergebnis im Dictionary des Slots `Grid.sieved` gespeichert. Innerhalb von `Grid` zählt eine Variable hoch, wie oft bereits gesiebt wurde; dieses wird als key für das dict genutzt.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =1)
g.sieve(stepSize = 3, offsetIndex =1, discardOffset= False)
for key in g.sieved:
    print(key)

##### OUTPUT
#####
#> sieved_0
#> sieved_1

```

Zusätzlich erlaubt discardOffset die sonst übersprungen Indices in das Resultat mit aufzunehmen.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =1)
g.sieve(stepSize = 3, offsetIndex =1, discardOffset= False)
for s in g.sieved:
    print(g.sieved[s]['sievedGrid'])

##### OUTPUT
#####
#> Grid(size=7, range=(1, 19), preview=[1, 4, 7, 10, 13, 16, 19])
#> Grid(size=8, range=(0, 19), preview=[0, 1, 4, 7, 10, 13, 16, 19])

```

Mittels inverse wird das Sieb umgekehrt. In dieser komplementären Filtrierung werden also alle Werte ausgegeben, die nicht im Sieb hängenbleiben.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =1)
g.sieve(stepSize = 3, offsetIndex =1, inverse=True)
for s in g.sieved:
    print(g.sieved[s]['sievedGrid'])

##### OUTPUT
#####
#> Grid(size=7, range=(1, 19), preview=[1, 4, 7, 10, 13, 16, 19])
#> Grid(size=12, range=(2, 18), preview=[2, 3, 5, 6, 8, 9, 11, 12, 14, 15, 17, 18])

```

reverse wiederum erlaubt die Filtrierung in umgekehrter Reihenfolge durchzuführen. Da das Umkehren des Inputs vor der Offset-Operation erfolgt, lassen sich damit Filtrierungen erzeugen, die nicht auf das Ende eines Grid angewendet werden.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =5, discardOffset = False)
g.sieve(stepSize = 3, offsetIndex =5, discardOffset = False, reverse=True)
for s in g.sieved:
    print(g.sieved[s]['sievedGrid'])

##### OUTPUT
#####
#> Grid(size=10, range=(0, 17), preview=[0, 1, 2, 3, 4, 5, 8, 11, 14, 17])
#> Grid(size=10, range=(2, 19), preview=[2, 5, 8, 11, 14, 15, 16, 17, 18, 19])

```

Mittels des Parameters rotation lässt sich eine weitere Offset-Operation durchführen, nachdem offsetIndex und reverse angewendet wurden.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =5, discardOffset=False)
g.sieve(stepSize = 3, offsetIndex =5, discardOffset=False, rotation=1)

```

```

g.sieve(stepSize = 3, offsetIndex =5, discardOffset=False, reverse=True)
g.sieve(stepSize = 3, offsetIndex =5, discardOffset=False,
reverse=True,rotation=1)

for s in g.sieved:
    print(s," : ",g.sieved[s]['sievedGrid'])

##### OUTPUT
#####
#> sieved_0 : Grid(size=10, range=(0, 17), preview=[0, 1, 2, 3, 4, 5, 8, 11,
14, 17])
#> sieved_1 : Grid(size=10, range=(0, 18), preview=[0, 1, 2, 3, 4, 6, 9, 12,
15, 18])
#> sieved_2 : Grid(size=10, range=(2, 19), preview=[2, 5, 8, 11, 14, 15, 16,
17, 18, 19])
#> sieved_3 : Grid(size=10, range=(1, 19), preview=[1, 4, 7, 10, 13, 15, 16,
17, 18, 19])

```

Das dict enthält außerdem alle Informationen, über das angewendete Sieb.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =5)
key = 'sieved_0'
sieved = g.sieved[key]
print(key)
for param in sieved:
    print(f"{param}: {sieved[param]}")

##### OUTPUT
#####
#> sieved_0
#> sievedGrid: Grid(size=5, range=(5, 17), preview=[5, 8, 11, 14, 17])
#> stepSize: 3
#> offsetIndex: 5
#> discardOffset: True
#> inverse: False
#> reverse: False
#> rotation: 0

```

5.4 Sampling

Neben dem Sieben ist Sampling eine weitere Möglichkeit, Elemente aus einem Grid auszuwählen. Sampling ruft die `random.sample()`-Funktion auf und wählt zufällig `numElements` Elemente aus dem Grid aus und erzeugt ein neues Grid mit dieser Auswahl.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
for _ in range(5):
    g.sample(5)
for i in range(5):
    print(g.sampled['sampled_'+str(i)])

```


6. TimeGrid

Ein TimeGrid dient dazu Daten in der Zeit zu quantisieren oder eine Menge von Zeitdaten zu generieren. Dies ist zuallererst mit den Grid-Konstrukturen möglich.

6.1 Vererbte Grid-Konstrukturen

Die implementierten vererbten Konstrukturen sind die direkte Erzeugung via *iterable*, *.series()* oder *.interpolate()*. Die anderen Konstrukturen sind zur Zeit noch nicht vollständig implementiert und sollten nur mit Vorsicht verwendet werden.

6.1.1 Direktes Initialisieren

Ein TimeGrid kann direkt mit Sekundenwerten initialisiert werden.

```
from GreasyPidgin.TimeGrid import TimeGrid

t = TimeGrid([0,1,2,3,4.5], bpm = 111)
print(t)

##### OUTPUT
#####
#> TimeGrid
#> sorted      = [0, 1, 2, 3, 4.5],
#> durationSec = 4.5,
#> durationBeats = 8.325,
#> BPM         = 111,
#> size        = 5,
#> layers:
```

6.1.2 Lineare Konstruktion

Via *.series(startTimeSec, stepSizeSec, size)*

```
from GreasyPidgin.TimeGrid import TimeGrid

t = TimeGrid.series(1, 1, 10, bpm = 111)
print(t)

##### OUTPUT
#####
#> TimeGrid
#> sorted      = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
#> durationSec = 10.0,
#> durationBeats = 18.5,
#> BPM         = 111,
#> size        = 10,
#> layers:
```

6.1.3 Lineare Interpolation

Via *.linearInterpolation(startTimeSec, endTimeSec, size)*

```
from GreasyPidgin.TimeGrid import TimeGrid

t = TimeGrid.linearInterpolation(startTimeSec=0, endTimeSec= 2, size= 11, bpm
=111)
print(t)

##### OUTPUT
#####
#> TimeGrid
#> sorted      = [0.0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.4, 1.6, 1.8, 2.0],
#> durationSec = 2.0,
#> durationBeats = 3.7,
#> BPM         = 111,
#> ticksPerBeat = 480,
```

```
#> size = 11,
#> layers:
```

6.1.4 Nicht vollständig

```
from GreasyPidgin.TimeGrid import TimeGrid
```

```
tr = TimeGrid.rand(10, 0, 10)
tg = TimeGrid.geom(1, 1.1, 10)
tf = TimeGrid.fib(10, 1, 2)
tl = TimeGrid.fill(10, lambda i: 2**i)
```

6.2 Mikrotime-basierte Konstruktoren

Die zwei vorrangigen Arten ein TimeGrid zu erzeugen sind allerdings die Beat-bezogenen Methoden `.fromSubdivision()` und `.concatenateRQQEvents()`. Beide Methoden folgen der Idee, eine Makrozeit in eine Mikrozeit zu teilen.

6.2.1 Multi-Mikrotime Teilung

Mit der Methode `fromSubdivisions(subdivisions, durationBeats)` kann eine vorab bestimmte Anzahl von Beats gleichzeitig in mehrere Untereinheiten unterteilt werden. Für jede Unterteilung wird der Resultierende Wert in ein separates Layer geschrieben, als auch dem TimeGrid selbst beigefügt. Dauern und Endpunkte, sowohl in Sekunden als auch Beats, werden automatisch berechnet.

```
from GreasyPidgin.TimeGrid import TimeGrid
```

```
t = TimeGrid.fromSubdivisions(subdivisions=[3,4], durationBeats=10)
print(t)
```

```
##### OUTPUT
#####
#> TimeGrid
#> sorted = [0.0, 0.25, 0.333333, 0.5, 0.666667, 0.75, 1.0, 1.25,
1.333333, 1.5, 1.666667, 1.75],
#> durationSec = 1.75,
#> durationBeats = 1.75,
#> BPM = 60.0,
#> ticksPerBeat = 480,
#> size = 12,
#> layers:
#> / 3 : [0.0, 0.333333, 0.666667, 1.0] ...
#> / 4 : [0.0, 0.25, 1.0, 0.5] ...
```

6.2.2 RQQ Teilung

RQQ ist eine Rhythmus-Notation, die von William “Bill” Gardner Schottstaedt als Teil von *Patchwork* / *OpenMusic* entwickelt wurde.¹² Die Idee hinter RQQ ist, dass eine beliebige verschachtelte Rhythmusstruktur mittels simpler Proportionen abgebildet werden kann. In dieser Python-Implementation kann ein RQQ-Objekt entweder alleinstehend verwendet werden oder als Konstruktor für ein TimeGrid.

Nehmen wir an, eine 4telnote entspricht einem Beat, so können wir eine 8eltriole wie folgt darstellen:

```
from GreasyPidgin.TimeGrid import RQQ
```

```
print(RQQ(1, [1,1,1]))
```

```
##### OUTPUT
#####
#> RQQ
```

¹²Originalquellen: <http://recherche.ircam.fr/equipes/repmus/PatchWork/>, <https://github.com/openmusic-project>; Implementation von Michael Edwards in `slippery` klicken: <https://michael-edwards.org/sc/manual/rhythms.html#tuplets>


```
#> durationBeats      : 1
#> subdivisions       : [1, 1, 1]
#> bpm                : 81.0
#> startOffsetBeats   : 0.0
#> startsBeats        : [0.0, 0.3333333333333333, 0.6666666666666666]
```

Mittel des Rest-Objekts¹³ können Pausen hinzugefügt werden. Eine 4telnote gefolgt von einer 8elpause und ein 8elnote sieht wie folgt aus:

```
from GreasyPidgin.TimeGrid import RQQ,R, Rest

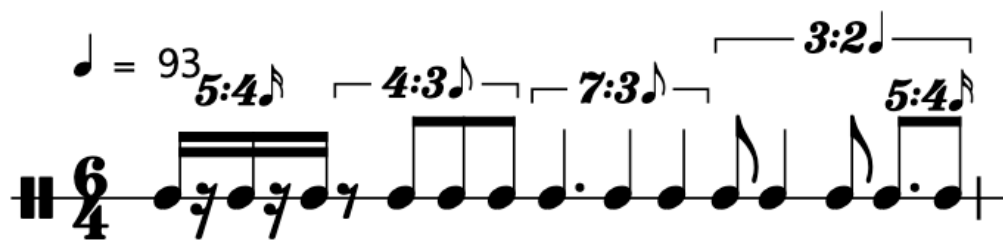
print(RQQ(2, [2,Rest(1),1]))
print("Rest == R: ",Rest(1) == R(1))
##### OUTPUT
#####
#> RQQ
#> durationBeats      : 2
#> subdivisions       : [2, 1.0, 1]
#> bpm                : 60.0
#> startOffsetBeats   : 0.0
#> startsBeats        : [0.0, 1.5]
#
#> Rest == R:  True
```

So können nun beliebig komplizierte Verästelungen geschrieben werden.

```
from GreasyPidgin.TimeGrid import RQQ, R, TimeGrid

rqq = RQQ(6,[
    RQQ(1, [1,R(1),1,R(1),1]),
    RQQ(1.5, [R(1),1,1,1]),
    RQQ(1.5, [3,2,2]),
    RQQ(2, [
        RQQ(2,[1,2,1]),
        RQQ(1, [3,2])
    ])
])

t = TimeGrid.fromRQQ(rqq, bpm = 93)
print(t)
t.parseMidi()
```



6.3 Midi-Export

Wie im vorigen Beispiel bereits durchgeführt, kann mit der Methode `.parseMidi()` das Grid, sowie alle Layer als Multi-Track Midifile exportiert werden.

¹³Als Kurzschreibweise ist auch R möglich

7. PitchGrid

Ein weiteres spezialisiertes Grid ist das PitchGrid. Es dient dazu Rohdaten in ein tonales System zu überführen oder generell ein harmonisches System zu erzeugen. Neben den vererbten Konstruktoren gibt es vor allem die Möglichkeit auf bestehende Intonations und Skalensysteme via `.fromScaleMaskObj()` zuzugreifen. Alternativ erlaubt der Konstruktor `.fromCorrelation()` aus einer Reihe von Möglichen Kandidaten das Stimmungssystem auszuwählen, das am besten auf die angegebenen Tönhöhen passt.

7.1 Stimmungssysteme

In der Datei `TuningSystemAndScaleMask.py` findet sich eine Sammlung von Skalen und Masken, mit denen sich eine Vielzahl verschiedener hamonischer Systeme abbilden lässt.

7.1.1 Oktavierende Stimmungssysteme

Die meisten musikalisch verwendeten Stimmungs- und Skalensystem bauen darauf auf, dass sie oktavtransponierend sind, das heißt, dass jede Tonhöhe potenziell eine beliebige Anzahl von Oktaven rauf und runter transponiert werden kann ohne den Charakter der Skala zu verändern.

7.1.1.1 Gleichschwebend

Als gleichschwebend bezeichnet man Stimmungssysteme in denen die Verhältnisse zwischen benachbarten Frequenzen immer die selben sind. Das in der westlichen Musik am weitesten verbreitete und standardisierte System ist die gleichschwebende Unterteilung der Oktave in 12 Schritte. Auf english: *12 Step Equal Distance Octave* oder kurz *edo12*. Digital ist allerdings die Unterteilung der Oktave in eine beliebige Anzahl von Schritten möglich. GreasyPidgin bietet standardmäßig nicht nur *edo12*, sondern zusätzlich *edo7*, *edo11*, *edo53* sowie *edo72*.

7.1.1.2 Nicht gleichschwebend - aber 12-tönig

Historisch gesehen stellen gleichschwebende System allerdings eine Minderheit dar. Als Alternative zu *edo12* beinhaltet GreasyPidgin daher die Möglichkeit Skalen in der folgenden Stimmungssystemen zu erzeugen: *werckmeister1*¹⁴, *werckmeister2*¹⁵, *werckmeister3*¹⁶, *werckmeister4*¹⁷, *kirnberger2*¹⁸, *kirnberger3*¹⁹, *meantoneQuarterComma*²⁰, *pythagorean*²¹.

Aus dem Bereich der *Just Intonation*-Systeme ist bislang ausschließlich *Zeta12*²² implementiert.

7.1.1.3 Weder gleichschwebend, noch 12-tönig

Neben diesen historischen europäischen Intonationssystemen ist mit *shruti22* auch die Grundlage für Skalen nach dem Vorbild von Ragas aus der karnatischen Tradition implementiert.²³

In Zukunft sollen auch noch weitere Systeme implementiert werden, wie zum Beispiel Pélog, Slendro, PartchJustIntonation und andere.²⁴

¹⁴Bezeichnet nach: http://www.groenewald-berlin.de/text/text_T016.html

¹⁵Bezeichnet nach: http://www.groenewald-berlin.de/text/text_T017.html

¹⁶Bezeichnet nach: http://www.groenewald-berlin.de/text/text_T018.html

¹⁷Bezeichnet nach: http://www.groenewald-berlin.de/text/text_T019.html

¹⁸nach <https://www.lehrklaenge.de/PHP/Tonsystem/KirnbergerStimmung.php>

¹⁹ebenda

²⁰https://de.wikipedia.org/wiki/Mitteltönige_Stimmung

²¹Pythagoreische Stimmung

²²Ratios von <https://en.xen.wiki/w/Zeta12>

²³siehe: <https://www.carnaticcorner.com/articles/sruthis.html> alternativ ist eine Erweiterung möglich: https://en.xen.wiki/w/A_shruti_list

²⁴Bei Interesse zur eigenen Implementation und Mitwirkung bietet das XenharmonicWiki einen guten Recherche-Einstieg: https://en.xen.wiki/w/Gallery_of_12-tone_just_intonation_scales

7.1.2 Nicht-oktavierende Stimmungssysteme

z.B.: Bohlen-Pierce ; bisher nicht implementiert

7.2 Skalen

GreasyPidgin erlaubt dabei eine hohe theoretische Flexibilität. Die bisher implementierten Skalen sind:

7.2.1 12-Ton-Basis

7.2.1.1 Kirchentonarten

- ionian, dorian, phrygian, lydian, mixolydian, aeolian, locrian

7.2.1.2 Kirchentonarten in harmonisch Moll

- harmonicMinor, locrianSharp6, ionianSharp5, dorianSharp4, phrygianDominant, lydianSharp2, ultraLocrian

7.2.1.3 Kirchentonarten in melodisch Moll

- melodicMinor, dorianFlat2, lydianAugmented, lydianDominant, mixolydianFlat6, locrianSharp2, altered

7.2.1.4 Modi mit begrenzten Transpositionsmöglichkeiten

- chromatic, wholeTone, halfToneWholeTone, messiaen3, messiaen4, messiaen5, messiaen6, messiaen7

7.2.1.5 Pentatoniken

- majorPentatonic, minorPentatonic, suspendedPentatonic, bluesMinorPentatonic, bluesMajorPentatonic

7.2.2 22-Shruti-Basis

7.2.2.1 Skalen nach hindustanischem Vorbild²⁵

- abhogi, mohanam, hamsadhwani, hindolam, shuddhaSaveri, madhyamavati, khamas, revati, kalyani, todi, bhairavi, shankarabharanam

7.2.2.2 Skalen nach karnatischem Vorbild

- n.n.²⁶

7.2.3 EDO53-Basis

7.2.3.1 Türkische Maqams

- n.n.

7.2.4 EDO72-Basis²⁷

7.2.4.1 Iranische Dastgāhs und Āwāz

- n.n.

²⁵Wichtige Anmerkung für alle Weißbrote: Es sind ausdrücklich Skalen nach dem *VORBILD* von Ragas, da der Begriff Raga mehr als nur eine Abfolge von Tonhöhen bezeichnet. Möchte man in diesem Klangraum arbeiten, empfehle ich Kontakt zu Musiker!nnen aufzunehmen, die sich in diesen Traditionen bewegen und mit denen man gemeinsam ein Verständnis für diese Klangsprache entwickelt. Das gleiche gilt auch für die weiteren Systeme von *Maqams* und *Dastgāhs*.

²⁶To be implemented: [Melakarta#Table of Melakarta ragas](#)

²⁷Zur Zeit ist noch nicht klar, ob und in welchem EDO-System ich die arabischen Maqams implementieren will. Unter https://en.xen.wiki/w/Middle-Eastern_music finden sich neben edo72 verschiedene Möglichkeiten.

7.3 Konstruktoren

7.3.1 Skalenerzeugung

.fromScaleInTUNING_SYSTEMS() bietet die Möglichkeit via Schlagwort eine Skala zu erzeugen. Der Parameter maskKey erwartet einen Skalennamen²⁸, tuningSystemKey erwartet ein zum Skalennamen passendes Stimmungssystem. Mit den Parametern lowestPitchMidi und highestPitchMidi lässt sich der maximale Ambitus des PitchGrid bestimmen.

```
from GreasyPidgin.PitchGrid import PitchGrid

p = PitchGrid.fromScaleInTUNING_SYSTEMS(
    maskKey= 'chromatic',
    tuningSystemKey= "edo12",
    lowestPitchMidi=45,
    highestPitchMidi=55)
print(p)

##### Output
#####
#> PitchGrid
#>   size      : 11,
#>   range      : 45.0 to 55.0,
#>   tuning      : 'edo12',
#>   scale       : 'chromatic',
#>   pitchClass : [0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0,
11.0],
#>   preview     : [45.0, 46.0, 47.0, 48.0, 49.0, 50.0, 51.0, 52.0, 53.0,
54.0 ...
```

Für alle Stimmungssysteme, die nicht edo12 oder Vielfache dessen (edo24, edo72, etc.) sind, ist es notwendig, den Startpunkt tuningSystemRoot für die Berechnung des Systems festzulegen.

```
from GreasyPidgin.PitchGrid import PitchGrid

p60 = PitchGrid.fromScaleInTUNING_SYSTEMS(tuningSystemKey=
"edo7",tuningSystemRootMidi = 60)
p59 = PitchGrid.fromScaleInTUNING_SYSTEMS(tuningSystemKey=
"edo7",tuningSystemRootMidi = 59)
print(p60)
print(p59)
# p59.parseMidi("/tmp/root59_")
# p59.parseMidi("/tmp/root60_")

# same PitchClass (edo7) but different resulting pitches due to different
# calculation start. The default state for all systems is "chromatic" but
# only
# edo12 produces a real chromatic scale. For all other tuning systems
# "chromatic" tries to find the closest pitch in the system that resembles a
# edo12 chromatic pitch. This doesn't work for shruti based systems!

##### Output
#####
# PitchGrid
#   size      : 8,
#   range      : 60.0 to 72.0,
#   tuning      : 'edo7',
#   scale       : 'chromatic',
#   pitchClass : [0.0, 1.71, 3.43, 5.14, 6.86, 8.57, 10.29],
#   preview     : [60.0, 61.71, 63.43, 65.14, 66.86, 68.57, 70.29, 72.0
```

²⁸Eine mask ist hier am besten mit den deutschen Wort *Schablone* zu übersetzen – ein Filter mittels welchem aus einem vollständigen chromatischen set nur die Töne ausgewählt werden, die für die Skala benötigt werden.

```
# PitchGrid
#   size      : 7,
#   range     : 60.71 to 71.0,
#   tuning    : 'edo7',
#   scale     : 'chromatic',
#   pitchClass : [0.0, 1.71, 3.43, 5.14, 6.86, 8.57, 10.29],
#   preview   : [60.71, 62.43, 64.14, 65.86, 67.57, 69.29, 71.0]
```

Noch deutlicher fällt dies bei historischen westlichen Stimmungen ins Gewicht. So lässt sich eine dorische Tonleiter in mitteltöniger Stimmung mit dem Grundton C sowohl in einem System konstruieren, das von C ausgehend konstruiert ist, oder zum Beispiel von Cis aus.

```
from GreasyPidgin.PitchGrid import PitchGrid
```

```
pg60 = PitchGrid.fromScaleInTUNING_SYSTEMS(
    'dorian',
    tuningSystemKey='meantoneQC',
    tuningSystemRootMidi=60,
    scaleRootMidi=60,
    highestPitchMidi=73
)
```

```
pg61 = PitchGrid.fromScaleInTUNING_SYSTEMS(
    'dorian',
    tuningSystemKey='meantoneQC',
    tuningSystemRootMidi=61,
    scaleRootMidi=60,
    highestPitchMidi=73
)
```

```
print(pg60)
print(pg61)
```

```
##### Output
#####
#> PitchGrid
#>   size      : 8,
#>   range     : 60.0 to 72.0,
#>   tuning    : 'meantoneQC',
#>   scale     : 'dorian',
#>   pitchClass : [0.0, 1.93, 3.1, 5.03, 6.97, 8.9, 10.07],
#>   preview   : [60.0, 61.93, 63.1, 65.03, 66.97, 68.9, 70.07, 72.0

# PitchGrid
#>   size      : 8,
#>   range     : 60.12 to 72.12,
#>   tuning    : 'meantoneQC',
#>   scale     : 'dorian',
#>   pitchClass : [0.12, 1.76, 2.93, 4.86, 6.79, 9.14, 9.9],
#>   preview   : [60.12, 61.76, 62.93, 64.86, 66.79, 69.14, 69.9, 72.12]
```

7.4 Export

Um ein PitchGrid darzustellen, lässt es sich wie ein TimeGrid via `.parseMidi()` als MIDI-Datei exportieren²⁹. Für die Verwendung in DAWs werden die mikrotonalen Abweichungen in PitchBend-Werte umgerechnet, für einen Notation werden sie als lyrics an den Noten annotiert.

```
from GreasyPidgin.PitchGrid import PitchGrid
```

```
pg60 = PitchGrid.fromScaleInTUNING_SYSTEMS(
    'dorian',
    tuningSystemKey='meantoneQC',
```

²⁹Beim Export als Midi muss allerdings die PitchBendRange auf das MIDI-Instrument abgestimmt sein, auf dem es wiedergegeben wird! Siehe `TemporalObject.parseMidi()` um ggf. eine Kalibrierungsdatei zu erzeugen.

```
tuningSystemRootMidi=60,  
scaleRootMidi=60,  
highestPitchMidi=73  
)  
pg60.parseMidi("/tmp/mqc_dorian_tsRoot60")
```

Output

#####

#> Midi file saved to: /tmp/

mqc_dorian_tsRoot60PitchGrid_meantoneQC_dorian.mid



8. DynamicGrid

Ein DynamicGrid erlaubt das Rastern von Dynamikwerten und die Übersetzung von reellen Lautstärken von Instrumenten in Dezibel in Midi-Werte.

```
from GreasyPidgin.DynamicGrid import DynamicGrid

d1 = DynamicGrid()
for entry in d1.dynamicMapping:
    print(entry, d1.dynamicMapping[entry])
```


9. TemporalObject

9.1 Was ist ein TemporalObject?

Ein TemporalObject – kurz T0 – ist die Superklasse für alle Objekte, die in der Zeit organisiert werden können. Das T0 vererbt alle Eigenschaften an:

Phonon: Ein *Klangpartikel* – die kleinste reduzierbare Einheit von Klang.

Gesture: Eine *Klanggeste* – eine Kombination mehrerer Phonon-Objekte, die eine Sinneinheit bilden.

GreasyPidgin: *Toplevel-Struktur*, die eine algorithmische Generation und Analyse komplexer Strukturen aus Phonon, Gesture, Phrase und Section ermöglicht.

Weitere T0-Objekte befinden sich noch in der Entwicklungsphase. Zur Zeit geplant sind:

Phrase: Eine abgeschlossene *musikalische Mikrostruktur*, bestehend aus einem oder mehreren Phonon- und Gesture-Objekten.

Section: Eine abgeschlossene *musikalische Makrostruktur*, ein Abschnitt aus Phonon-, Gesture- und/oder Phrase-Objekten.

Photon: Lichtsteuerdaten (noch nicht implementiert).

Graphon: Videosteuerdaten (noch nicht implementiert).

9.2 Aufbau

Ein TemporalObject besteht aus drei Bereichen:

Zeit – startTimeSec, durationSec, endTimeSec. Für Container wird die Dauer automatisch aus den children berechnet.

Hüllkurven – envelopes: dict[str, Envelope]. Zeitlich variierende Eigenschaften des Objekts selbst, unabhängig vom Inhalt. Beispiele: 'vibrato', 'tremolo', 'opacity' für visuelle Objekte.

Daten – data: dict. Alle domänenspezifischen Informationen. Ein MIDI-Export liest z.B. data['pitchMidi'], data['dynamicdB'] und data['bpm']. Ein Video-Export würde data['file'] und data['framerate'] lesen. Das T0 selbst stellt keine Annahmen über den Inhalt von data an.

9.3 Ein TemporalObject erzeugen

Ein T0 wird immer mit Startzeit und Dauer initialisiert. Alle weiteren Parameter sind keyword-only:

```
from GreasyPidgin.TemporalObject import TemporalObject
from GreasyPidgin.Envelope import Envelope

# Einfaches Blatt-Objekt mit Daten
leaf = TemporalObject(
    0.0, 10.0,
    data={
        "pitchMidi": 60.0,
        "dynamicdB": 80.0,
        "bpm": 120,
        "ticksPerBeat": 480,
    })

# Mit Hüllkurven
leaf2 = TemporalObject(
    1.0, 3.1415,
    envelopes={"vibrato": Envelope([0, 1, 0, 1, 0, 1, 0])},
    annotations=[("Hello World!", 0.0)],
    data={"pitchMidi": 64.0, "dynamicdB": 90.0, "bpm": 120},
```

```

)

# Container – Dauer wird automatisch aus children bestimmt
container = TemporalObject(
    0.0,
    children=[leaf, leaf2],
)

print(container)

##### OUTPUT
#####
#> TemporalObject(id=2, start=0.000, dur=4.142, end=4.142, children=2)

```

Wird ein T0 mit children initialisiert, so werden Dauer und Endzeitpunkt automatisch anhand der Zeitwerte der children bestimmt. Die Zeiten der children sind immer *relativ* zur Startzeit des Containers.

9.4 Ein TemporalObject exportieren

Export geschieht über separate Exporter-Klassen aus `Exporters.py`. Das T0 selbst kennt keine Exportformate – es gibt sich nur an einen Exporter weiter:

```

from GreasyPidgin.TemporalObject import TemporalObject
from GreasyPidgin.Exporters import MidiExporter, JsonExporter, CsvExporter

t1 = TemporalObject(0.0, 2.0, data={"pitchMidi": 60.0, "dynamicdB": 80.0,
    "bpm": 120})
t2 = TemporalObject(2.0, 2.0, data={"pitchMidi": 64.0, "dynamicdB": 75.0,
    "bpm": 120})
to = TemporalObject(0.0, children=[t1, t2])

# MIDI
to.export(MidiExporter("/tmp/out.mid"))

# JSON – komplette Baumstruktur
to.export(JsonExporter("/tmp/out.json"))

# CSV – Baumstruktur als id/parentId Tabelle
to.export(CsvExporter("/tmp/out.csv"))

##### OUTPUT
#####
#> MidiExporter: saved -> /tmp/out.mid
#> JsonExporter: saved -> /tmp/out.json
#> CsvExporter: saved -> /tmp/out.csv

```

Oder alle auf einmal:

```

for exporter in [
    MidiExporter("/tmp/out.mid"),
    JsonExporter("/tmp/out.json"),
    CsvExporter("/tmp/out.csv"),
]:
    to.export(exporter)

```

9.5 MIDI-Export im Detail

Der `MidiExporter` liest folgende Schlüssel aus `leaf.data`:

Key	Typ	Bedeutung	Default
pitchMidi	float	MIDI-Notenwert (Nachkommastellen = Mikrotöne)	0.0
dynamicdB	float	Lautstärke in dB	100.0
bpm	float	Tempo	60.0

Key	Typ	Bedeutung	Default
ticksPerBeat	int	MIDI-Auflösung	480
annotations	list	[(offsetSec, text), ...] arrow.r Lyric-Events	[]

Zusätzlich werden alle `leaf.envelopes` als MIDI-CC- bzw. Pitchwheel-Daten exportiert (sofern `parseEnvelopes=True`).

9.5.1 Parameter des MidiExporters

```
MidiExporter(
    path           = "/tmp/out.mid",
    trackName      = "track",
    parseEnvelopes = True,
    perNoteTracks  = False, # True: überlappende Noten auf separate Tracks
    pitchBendRange = 4096,  # Ticks pro Halbton; 4096 × 2 = voller Wheel-
    Range
    channel        = 0,      # MIDI-Kanal (0-indiziert)
)
```

Danger

Die `pitchBendRange` muss auf den Synthesizer abgestimmt sein! Die meisten Synthesizer haben entweder 2^{10} oder 2^{12} Auflösung.

Synth-Plugin (Ableton Live)	Pitch Bend Range
Analog	2^{12}
Drift	2^{12}
Electric	2^{10}
Meld	2^{10}
Operator	2^{10}
Tension	2^{12}
Wavetable	2^{12}

Kalibrierungsdateien erzeugen:

```
from GreasyPidgin.TemporalObject import TemporalObject
from GreasyPidgin.Exporters import MidiExporter

container = TemporalObject(0.0)
for i in range(11):
    container.addChild(TemporalObject(
        i / 2, 0.5,
        data={"pitchMidi": 60 + (i / 10), "dynamicdB": 100.0, "bpm": 60},
    ))

for pbRange in [2**10, 2**12]:
    container.export(MidiExporter(
        f"/tmp/pitchBendConfig_{pbRange}.mid",
        pitchBendRange=pbRange,
    ))
```

9.5.2 Mikrotöne

```
from GreasyPidgin.TemporalObject import TemporalObject
from GreasyPidgin.Exporters import MidiExporter

to = TemporalObject(0.0, 10.0, data={
    "pitchMidi": 60.5, # Viertelton über c4
    "dynamicdB": 80.0,
    "bpm": 60,
```

```
})
```

```
to.export(MidiExporter("/tmp/quartertone.mid"))
```

Der MidiExporter berechnet automatisch den nächsten 12-EDO-Halbtton als MIDI-Notenwert und schreibt die Differenz als Pitchwheel-Envelope.

9.5.3 Polyphonie und perNoteTracks

Enthält ein T0 gleichzeitige Ereignisse mit verschiedenen mikrotonalen Abweichungen, muss perNoteTracks=True gesetzt werden. Andernfalls überschreiben sich die Pitchwheel-Befehle gegenseitig.

```
from GreasyPidgin.TemporalObject import TemporalObject
from GreasyPidgin.Exporters import MidiExporter

t1 = TemporalObject(0.0, 10.0, data={"pitchMidi": 60.0, "dynamicdB": 80.0,
    "bpm": 60})
t2 = TemporalObject(1.0, 10.0, data={"pitchMidi": 69.69, "dynamicdB": 80.0,
    "bpm": 60})
container = TemporalObject(0.0, children=[t1, t2])

# Pitchwheel-Konflikte möglich:
container.export(MidiExporter("/tmp/singleTrack.mid"))

# Separate Tracks pro Note – sicher:
container.export(MidiExporter("/tmp/perNoteTracks.mid", perNoteTracks=True))
```

9.6 Baumstruktur

Ein T0 kann beliebig tief verschachtelte children enthalten. Beim Exportieren wird der Baum automatisch zu einer flachen Liste von Blatt-Objekten mit absoluten Zeitwerten aufgefaltet (flatten()), ohne dass das Original verändert wird.

```
from GreasyPidgin.TemporalObject import TemporalObject

t1 = TemporalObject(0.0, 10.0, data={"pitchMidi": 60.0, "bpm": 120})
t2 = TemporalObject(5.0, 10.0, data={"pitchMidi": 64.0, "bpm": 120})
t3 = TemporalObject(0.0, children=[t1, t2])
t4 = TemporalObject(3.0, children=[t1, t2, t3])
t5 = TemporalObject(5.0, children=[t1, t2, t3, t4])

# Alle Blatt-Objekte mit absoluten Zeiten
for leaf in t5.flatten():
    print(leaf)
```

Die Zeiten der children sind immer relativ zum jeweiligen Container. flatten() addiert die Offsets rekursiv und gibt Kopien zurück — das Original bleibt unberührt.

10. Phonon

10.1 Was ist ein Phonon?

Ein Phonon ist ein *Klangpartikel* – die kleinste beschreib- und reduzierbare Einheit von Klang. Der Komplexität sind dabei keine Grenzen gesetzt.

Ein Phonon erweitert das TemporalObject um musikalisches Vokabular: Tonhöhen in verschiedenen Einheiten, Lautstärke, Akkord- und Sequenzstrukturen. Die musikalischen Informationen werden intern verarbeitet und erst beim Export über `unfold()` in konkrete TemporalObject-Blätter aufgelöst.

10.2 Ein Phonon erzeugen

10.2.1 Sekunden oder Beats

Ein Phonon muss wie ein TemporalObject immer mit einer Startzeit und einer Dauer initialisiert werden. Im musikalischen Kontext kann die Zeit entweder in 'seconds' / 's' (Standard) oder 'beats' / 'b' angegeben werden:

```
from GreasyPidgin.Phonon import Phonon
```

```
ps = Phonon(1, 10, timeUnit='s')
pb = Phonon(1, 10, timeUnit='b', bpm=161)
```

```
print(ps)
print(pb)
```

```
##### OUTPUT
```

```
#####
```

```
# Phonon(
#   id          : 0
#   start       : 1.000s (1.000 beats)
#   duration    : 10.000s (10.000 beats)
#   bpm        : 60.0
#   chordOrSeq  : seq
#   allPitches  : [0.0]
#   lowest      : [0.0]
#   highest     : [0.0]
#   voices      : 1
#   envelopes   : ['dynamic']
#   children    : 0
# )
```

```
# Phonon(
#   id          : 1
#   start       : 0.373s (1.000 beats)
#   duration    : 3.727s (10.000 beats)
#   bpm        : 161
#   ...
# )
```

10.2.2 Tonhöhe

Die wesentliche Erweiterung gegenüber dem TemporalObject ist, dass Tonhöhen in verschiedenen Einheiten angegeben werden können: 'midi' (Standard), 'hz' oder 'spn'³⁰.

```
from GreasyPidgin.Phonon import Phonon
```

```
pMidi      = Phonon(1, 1, pitch=68)
pMidiMicro = Phonon(1, 1, pitch=68.45) # Mikrotöne als Nachkommastellen
pSPN       = Phonon(1, 4, 'af4', pitchUnit='spn')
pHz        = Phonon(1, 2, 440, pitchUnit='hz')
```

³⁰Scientific Pitch Notation, siehe `Pitch.py`, `02_Pitch.md` oder **Kapitel 2: Pitch**.


```
print("MIDI-Wert für pSPN:", pSPN.allPitches[0])
print("MIDI-Wert für pHz:", pHz.allPitches[0])
```

```
##### OUTPUT
#####
# MIDI-Wert für pSPN: 68.0
# MIDI-Wert für pHz: 69.0
```

10.2.3 Lautstärke

dynamic kann entweder ein einzelner Wert oder zeitlich variierend sein:

```
from GreasyPidgin.Phonon import Phonon
from GreasyPidgin.Envelope import Envelope

# Einzelwert – wird als Velocity beim Export genutzt (dynamicUnit='midi' oder 'dB')
pFlat = Phonon(0, 4, 60, dynamic=80, dynamicUnit='midi')
pFlat2 = Phonon(0, 4, 60, dynamic=90.0, dynamicUnit='dB')

# Liste – gleichmäßig über die Dauer verteilt (Crescendo)
pCresc = Phonon(0, 4, 60, dynamic=[20, 60, 127], dynamicUnit='midi')

# Envelope – volle Kontrolle über die zeitliche Form
env = Envelope([(0, 20), (2, 127), (4, 60)])
pEnv = Phonon(0, 4, 60, dynamic=env)
```

Note

Ist dynamic eine Liste oder Envelope, wird die Hüllkurve als envelopes['aftertouch'] gespeichert und beim MIDI-Export als Channel-Aftertouch exportiert. Ein einzelner Wert wird als data['dynamicdB'] gespeichert und als Velocity der Note verwendet.

10.3 Tonhöhenstruktur

Ein Phonon kennt drei Tonhöhenstrukturen, gesteuert über chordOrSeq:

10.3.1 'seq' / 's' — Monophones Sequenz (Standard)

Eine Abfolge von Tonhöhen, gleichmäßig über die Dauer verteilt. Sonderfall Länge 1 = eine einzige gehaltene Note.

```
from GreasyPidgin.Phonon import Phonon

pSingle = Phonon(1, 1, 'af4', pitchUnit='spn')
pList = Phonon(1, 1, ['af4'], pitchUnit='spn')
print(pSingle.allPitches == pList.allPitches,
      "– beide sind eine einelementige 'seq'")

pSeq = Phonon(1, 1, ['af4', 'b4', 'bf4', 'b4'], pitchUnit='spn')
print(pSeq)
```

```
##### OUTPUT
#####
# True – beide sind eine einelementige 'seq'
# Phonon(
#   id          : 2
#   allPitches   : [68.0, 70.0, 71.0]
#   lowest      : [68.0]
#   highest     : [71.0]
#   voices      : 1
#   ...
# )
```

10.3.2 'chord' / 'c' — Akkord

Alle Töne klingen gleichzeitig für die gesamte Dauer. Geeignet für Akkorde und Spektren.

```

from GreasyPidgin.Phonon import Phonon

pChladni = Phonon(
    2, 10,
    [320, 630, 733, 854, 1370, 1650, 1850, 3159],
    pitchUnit='hz',
    chordOrSeq='chord',
)
print(pChladni)

##### OUTPUT
#####
# Phonon(
#   id          : 0
#   allPitches   : [63.49, 75.21, 77.84, 80.48, 88.66, 91.88, 93.86, 103.13]
#   lowest      : [63.49]
#   highest     : [103.13]
#   voices      : 8
#   ...
# )

```

Figure 10.1: Beispiel-Spektrum aus Chladni-Platten-Resonanzen.

10.3.3 'chordSeq' / 'cs' — Akkordsequenz

Eine Folge von Akkorden, gleichmäßig über die Dauer verteilt. Die Anzahl der Stimmen darf zwischen den Akkorden variieren. Standardmäßig werden die Töne innerhalb jedes Akkords von unten nach oben sortiert (autoSortChords=True).

```

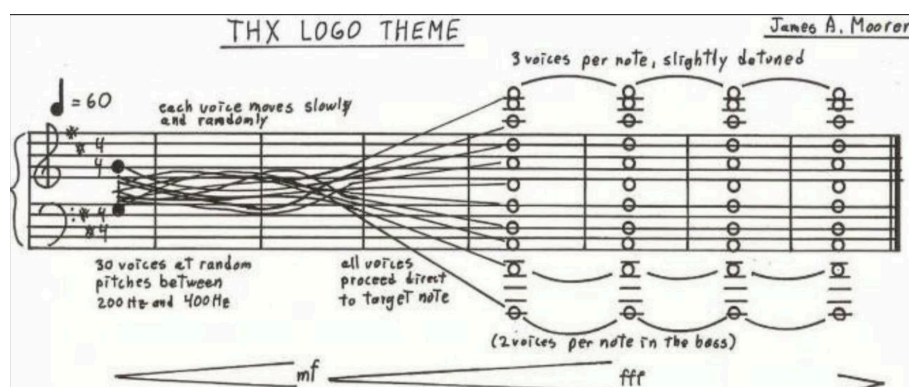
from random import randint
from numpy import clip
from GreasyPidgin.Phonon import Phonon
from GreasyPidgin.Pitch import spntof

# Zufällige Frequenzverläufe – wie das THX-Logo-Thema
stages = [[randint(200, 400) for _ in range(30)]]
for _ in range(8):
    newStage = [int(clip(f + randint(-20, 20), 200, 400)) for f in stages[-1]]
    stages.append(newStage)

finalChordSPN = [
    'd1', 'd1', 'd2', 'd2', 'd3', 'd3', 'd3',
    'a2', 'a2',
    'd4', 'd4', 'dtf4', 'dts4', 'd5', 'd5', 'dtf5', 'dts5', 'd6', 'd6', 'dtf6', 'dts6',
    'a4', 'atf4', 'ats4', 'a5', 'atf5', 'ats5',
    'fs5', 'fstf5', 'fst5',
]
stages.append([spntof(f) for f in finalChordSPN])

thx = Phonon(0, 29, stages, pitchUnit='hz', chordOrSeq='chordSeq')
print(thx)

```



10.4 Glissando

Ein Glissando wird als *analytische / deskriptive* Hüllkurve gespeichert — es beeinflusst den MIDI-Export nicht direkt, steht aber für Score-Rendering, Analyse oder eigene Exporter zur Verfügung.

```
from GreasyPidgin.Phonon import Phonon

p = Phonon(0, 4, 'c4', pitchUnit='spn')
p.glissando('c4', 'e4') # Glissando von C4 nach E4 über 4 Sekunden

print(p.envelopes['glissando']) # Envelope mit MIDI-Werten 60.0 → 64.0
```

10.5 unfold() — Tonhöhen auflösen

Bevor ein Phonon exportiert werden kann, müssen seine Tonhöhen in konkrete TemporalObject-Blätter aufgelöst werden. Das geschieht automatisch beim Aufruf einer Export-Methode, kann aber auch manuell ausgelöst werden:

```
p = Phonon(0, 10, ['c4', 'e4', 'g4'], pitchUnit='spn', chordOrSeq='seq')
p.unfold(noteSelectKey='lowest')

for child in p.children:
    print(child)
```

unfold() ist rekursiv — enthält ein Phonon weitere Phonon-Objekte als children, werden diese zuerst aufgelöst.

10.5.1 Optionen für noteSelectKey

Key	Bedeutung
'lowest'	Tiefste Note aus allen MIDI-Werten (Standard)
'highest'	Höchste Note
'mostCommon'	Häufigste Note
'leastCommon'	Seltenste Note
'all'	Alle Noten als Blockakkord, inkl. Dopplungen
'chord' / 'allSet'	Alle Noten als Blockakkord, ohne Dopplungen
'seq'	Eine Note pro Zeitslot (aus der ersten Pitch-Envelope)
'chordSeq'	Alle Stimmen aller Akkord-Zeitslots

10.6 Export

10.6.1 MIDI

```
from GreasyPidgin.Phonon import Phonon

p = Phonon(0, 10, [['af4', 'a4'], ['af4', 'gqs4'], ['af4', 'a4']],
             chordOrSeq='cs', pitchUnit='spn')

# Standard: tiefste Note
p.toMidi('/tmp/lowest.mid')

# Alle Akkorde auf separaten Tracks
p.toMidi('/tmp/chordSeq.mid', noteSelectKey='chordSeq', perNoteTracks=True)
```

10.6.2 Alle Formate

```
p.toMidi('/tmp/out.mid', noteSelectKey='chordSeq', perNoteTracks=True)
p.toXml('/tmp/out.xml', noteSelectKey='lowest')
p.toCsv('/tmp/out.csv', noteSelectKey='all')
p.toJson('/tmp/out.json', noteSelectKey='chord')
```

10.6.3 Beispiele für alle noteSelectKey-Optionen

Sequenz:

```
from GreasyPidgin.Phonon import Phonon

p = Phonon(0, 10,
    ['af4', 'a4', 'a4', 'gqs4', 'af4', 'a4'],
    chordOrSeq='s', pitchUnit='spn')

for key in
    ['seq', 'chordSeq', 'lowest', 'highest', 'mostCommon', 'leastCommon', 'allSet', 'chord']:
    p.toMidi(f'/tmp/seq_{key}.mid', noteSelectKey=key)
```

Akkordsequenz:

```
from GreasyPidgin.Phonon import Phonon

p = Phonon(0, 10,
    [['af4', 'a4'], ['a4', 'gqs4'], ['af4', 'a4']],
    chordOrSeq='cs', pitchUnit='spn')

for key in
    ['seq', 'chordSeq', 'lowest', 'highest', 'mostCommon', 'leastCommon', 'allSet', 'chord']:
    p.toMidi(f'/tmp/cs_{key}.mid', noteSelectKey=key)
```

Akkord:

```
from GreasyPidgin.Phonon import Phonon

p = Phonon(0, 10,
    ['af4', 'a4', 'a4', 'gqs4', 'af4', 'a4'],
    chordOrSeq='c', pitchUnit='spn')

for key in
    ['seq', 'chordSeq', 'lowest', 'highest', 'mostCommon', 'leastCommon', 'allSet', 'chord']:
    p.toMidi(f'/tmp/chord_{key}.mid', noteSelectKey=key)
```


11. Gesture

11.1 Was ist eine Gesture?

Eine Gesture ist eine *Klanggeste* - die kleinste beschreib- und reduzierbare Gruppierung mehrerer *Klangpartikel*. Der Komplexität sind dabei allerdings keine Grenzen gesetzt.

11.2 Eine Gesture erzeugen

11.2.1 Sekunden oder Beats

Ein Phonon muss wie ein TemporalObject immer mit einer Startzeit und einer Dauer initialisiert werden. Dabei ist es in einem jetzt musikalischen Kontext möglich die Zeit entweder in 'seconds' / 's' (default) oder 'beats' / 'b' zu setzen.

```
from GreasyPidgin.Phonon import Phonon

ps = Phonon(1,duration = 10,timeUnit='s')
pb = Phonon(1,duration = 10,timeUnit='b', bpm = 161)

print(ps)
print(pb)

##### OUTPUT
#####
# Phonon(
#   id      : 1
#   startSec : 1.000, durSec: 10.000, endSec: 11.000
#   pitchesMidi :
#     - all      : {0}
#     - highest   : [0]
#     - lowest    : [0]
#     - mostCommon : [0.0]
#     - leastCommon : [0.0]
#
# [0]
# Phonon(
#   id      : 2
#   startSec : 0.373, durSec: 3.727, endSec: 4.099
#   pitchesMidi :
#     - all      : {0}
#     - highest   : [0]
#     - lowest    : [0]
#     - mostCommon : [0.0]
#     - leastCommon : [0.0]
```

11.2.2 Tonhöhe

Die essentielle Erweiterung ist nun, dass Tonhöhen ergänzt werden können. Die Tonhöhen selbst können dabei entweder als 'midi', 'hz' oder 'spn'³¹ formatiert sein.

```
from GreasyPidgin.Phonon import Phonon

pSingleMidi = Phonon(1,1,pitch=68)
pMidiMicro = Phonon(1,1,pitch=68.45)

pFromSPN = Phonon(1,4,'af4', pitchUnit = 'spn')

pFromFreq = Phonon(1,2, 440, pitchUnit='hz')

if type(pSingleMidi) == Phonon and type(pMidiMicro) == Phonon:
    print("This is 'True' because pitchUnit='midi' is the default")

print("The midipitch for pFromSPN is: ",pFromSPN.allPitches[0])
```

³¹scientific pitch notation, siehe Pitch.py, 02_Pitch.md oder **Kapitel 2: Pitch**

```
print("The midipitch for pFromFreq is: ",pFromFreq.allPitches[0])
```

```
##### OUTPUT
#####
# This is 'True' because pitchUnit='midi' is the default
# The midipitch for pFromSPN is: 68.0
# The midipitch for pFromFreq is: 69.0
```

11.2.3 Tonhöhenstruktur

Ein Phonon kann drei verschiedenen Tonhöhenstrukturen nutzen:

- 'seq' / 's': Eine monophone Sequenz von Tonhöhen, inkl. des Spezialfalls Sequenzlänge 1 für nur eine Tonhöhe => *default*. In dieser Version muss die Abfolge von Tonhöhen in gleichen Abständen erfolgen.

```
from GreasyPidgin.Phonon import Phonon
```

```
pSingle = Phonon(1,1,pitch='af4',pitchUnit='spn',)
pList = Phonon(1,1,pitch=['af4'],pitchUnit='spn',)
print(pSingle.allPitches == pList.allPitches, ", because both Phonons are a
one-element 'seq' by default")
```

```
pSeq = Phonon(1,1,pitch=['af4', 'b4', 'bf4', 'b4'],pitchUnit='spn',)
print(pSeq)
```

```
##### OUTPUT
#####
# True , because both Phonons are a one-element 'seq' by default
# Phonon(
#   id          : 2
#   startSec     : 1.000, durSec: 1.000, endSec: 2.000
#   pitchesMidi  :
#     - all      : [68.0, 70.0, 71.0]
#     - highest  : [71.0]
#     - lowest   : [68.0]
#     - mostCommon : [71.0]
#     - leastCommon : [70.0]
```

- 'chord' / 'c': Ein einzelner Akkord bzw ein einzelnes komplexes Spektrum³².

```
from GreasyPidgin.Phonon import Phonon
```

```
pChladniSpectrum = Phonon(
    2,10,
    [320, 630, 733, 854, 1370, 1650, 1850, 3159],
    pitchUnit='hz',
    chordOrSeq='chord')
```

```
print(pChladniSpectrum)
```

```
##### OUTPUT
#####
# Phonon(
#   id          : 0
#   startSec     : 2.000, durSec: 10.000, endSec: 12.000
#   pitchesMidi  :
#     - all      : [63.486820576352436, 75.21417965835143,
77.83571609530983, 80.48079055314997, 88.66320557187876, 91.88268714730222,
93.86339810254819, 103.12671009866656]
#     - highest  : [103.12671009866656]
#     - lowest   : [63.486820576352436]
```

³²Bespiel-Spektrum entnommen aus: https://www.researchgate.net/publication/380052462_Chladni_Plate_and_Chladni_Patterns-A_Research_Review_of_Theory_Modelling_Simulation_and_Engineering_Applications

```
# - mostCommon : [63.486821]
# - leastCommon : [103.12671]
```

Figure 11.1: image("files/chladni01-52837f7106d97ee65f535a1543145201.png", width: 90%)
 image("files/chladni02-059f3b868f1f049b5d41cc3bdcddcccbe.png", width: 90%)

- 'chordSeq' / 'cs': Eine Akkordsequenz oder ein sich modulierendes Spektrum. Wie bei der einfachen seq wird auch hier angenommen, dass die zeitlichen Abstände zwischen den einzelnen Akkorden identisch sind. Grundeinstellung ist, dass die einzelnen Akkorde via `autoSortChords = True` vom tiefsten zum höchsten Ton sortiert werden.

```
from random import randint
from numpy import clip

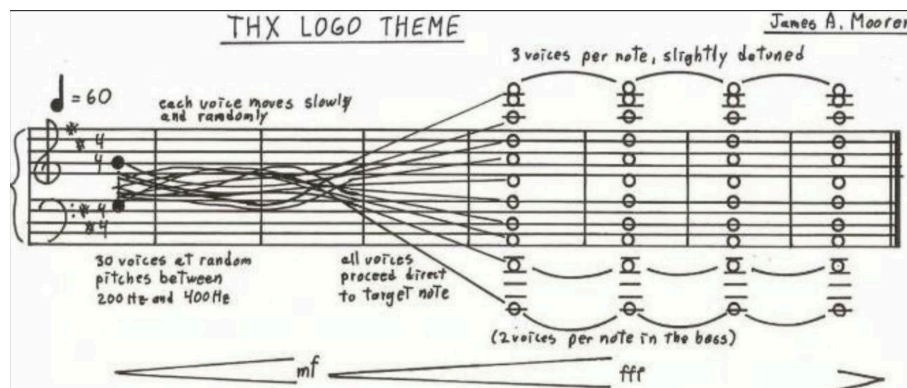
from GreasyPidgin.Phonon import Phonon
from GreasyPidgin.Pitch import spntof

stages = [[randint(200, 400) for _ in range(30)]]
for _ in range(8):
    newStage = [int(clip(f+randint(-20, 20), 200, 400)) for f in stages[-1]]
    stages.append(newStage)
finalChordSPN = [
    'd1', 'd1', 'd2', 'd2', 'd3', 'd3', 'd3',
    'a2', 'a2',
    'd4', 'd4', 'dtf4', 'dts4', 'd5', 'd5', 'dtf5', 'dts5', 'd6', 'd6', 'dtf6', 'dts6',
    'a4', 'atf4', 'ats4', 'a5', 'atf5', 'ats5',
    'fs5', 'fstf5', 'fst5']
stages.append([spntof(f) for f in finalChordSPN])

thxLogoTheme = Phonon(0, 29, stages, pitchUnit='hz', chordOrSeq='chordSeq')
print(thxLogoTheme)

# thxLogoTheme.parseMidi(noteSelectKey='chordSeq') # more on this later!

##### OUTPUT
#####
# Phonon(
#   id      : 0
#   startSec : 0.000, durSec: 29.000, endSec: 29.000
#   pitchesMidi :
#     - all      : [26.0, 38.0, ... ]
#     - mostCommon : [67.349958]
#     - leastCommon : [78.166667]
```



11.3 Export

Bisher ist, wie für das TemporalObject, nur der Export als MIDI-Datei möglich.

11.3.1 Midi

Mit der Methode `.parseMidi(folder, trackname, perNoteTracks, noteSelectKey)` kann ein Phonon als MIDI-Datei exportiert werden. Dabei gibt es verschiedenen Möglichkeiten, die Komplexität eines Phonon abzubilden. Als Standardeinstellung wird mit dem Parameter `noteSelectKey = 'lowest'` immer der tiefste vorhandene monophone Midiwert zum Export genutzt:

```
from GreasyPidgin.Phonon import Phonon
p = Phonon(0, 10, [['af4', 'a4'], ['af4', 'gqs4'], ['af4', 'a4']],
chordOrSeq='cs', pitchUnit='spn')
p.parseMidi('/tmp/default_is_')
print('\n\n')
p.parseMidi()

##### OUTPUT
#####
# Transformed pitch(es) to:
#   TemporalObject(id=0, start=0.000, dur=10.000, end=10.000, dyn=100.0 dB,
pitch=67.5, bpm=60.0, track=0)

# Midi file saved to: /tmp/default_is_lowest.mid
# Particle: 0 saved to /tmp/default_is_lowest.mid

# Transformed pitch(es) to:
#   TemporalObject(id=1, start=0.000, dur=10.000, end=10.000, dyn=100.0 dB,
pitch=67.5, bpm=60.0, track=0)

# Midi file saved to: /tmp/lowest.mid
# Particle: 0 saved to /tmp/lowest.mid
```

11.3.1.1 Optionen für noteSelectKey

Die weiteren Optionen sind: `'highest'`, `'mostCommon'`, `'leastCommon'`, `'all'`, `'allSet'` / `'chord'`, `'seq'` und `'chordSeq'`.

- `'highest'`: Die höchste Note aus allen enthaltenen Midiwerten.
- `'mostCommon'`: Die häufigste Note aus allen enthaltenen Midiwerten.
- `'leastCommon'`: Die seltenste Note aus allen enthaltenen Midiwerten.
- `'all'`: Alle Noten als Blockakkord, inkl Dopplungen.
- `'allSet'` / `'chord'`: Alle Noten als Blockakkord, aber ohne Dopplungen.
- `'seq'`: Ideal für homophone Tonhöhen-Bewegungen. Bei akkordischen Sequenzen wird Unterstimme ausgegeben. Alternativ kann beim initialisieren mit `autoSortChords = False` die erste erzeugte Sequenz exportiert werden.
- `'chordSeq'`: Alle Akkorde einer Sequenz, die Akkorde sind per se frei von Stimmkreuzungen. Um diese zu erlauben, muss beim initialisieren `autoSortChords = False` gesetzt werden.

Beispiel für eine Sequenz:

```
from GreasyPidgin.Phonon import Phonon
p = Phonon(
    0, 10,
    pitch=['af4', 'a4', 'a4', 'gqs4', 'af4', 'a4'],
    chordOrSeq='s', pitchUnit='spn')

p.parseMidi(noteSelectKey='seq')
p.parseMidi(noteSelectKey='chordSeq')
p.parseMidi(noteSelectKey='lowest')
p.parseMidi(noteSelectKey='highest')
p.parseMidi(noteSelectKey='mostCommon')
p.parseMidi(noteSelectKey='leastCommon')
```

```
p.parseMidi(noteSelectKey='allSet')
p.parseMidi(noteSelectKey='chord')
```

Beispiel für Akkordsequenzen:

```
from GreasyPidgin.Phonon import Phonon
p = Phonon(
    0, 10,
    pitch=[['af4', 'a4'], ['a4', 'gqs4'], ['af4', 'a4']],
    chordOrSeq='cs', pitchUnit='spn')

for noteSelectKey in
['seq', 'chordSeq', 'lowest', 'highest', 'mostCommon', 'leastCommon', 'allSet', 'chord']:
    p.parseMidi(noteSelectKey=noteSelectKey)
```

Beispiel für einen Akkord:

```
from GreasyPidgin.Phonon import Phonon
p = Phonon(
    0, 10,
    pitch=['af4', 'a4', 'a4', 'gqs4', 'af4', 'a4'],
    chordOrSeq='c', pitchUnit='spn')

for noteSelectKey in [
    'seq', 'chordSeq', 'lowest', 'highest',
    'mostCommon', 'leastCommon', 'allSet', 'chord'
]:
    p.parseMidi(noteSelectKey=noteSelectKey)
```


12. Zellulärer Automat à la Wolfram

Eine einführende Anleitung zur Musikgenerierung mit einem Wolfram-Elementarautomaten und GreasyPidgin.

Am Ende dieses Tutorials hast du eine MIDI-Datei mit einer Melodie, deren **Tonhöhen** aus dem sich entwickelnden Zustand eines zellulären Automaten stammen und deren **Rhythmik** durch die binären Zellen desselben Automaten bestimmt wird.

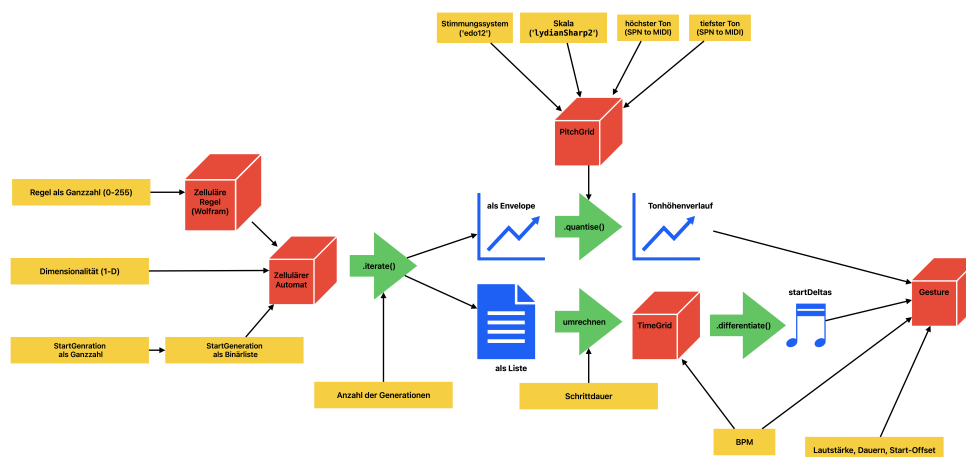
12.1 Was ist ein Zellulärer Automat?

Ein zellulärer Automat (ZA) ist ein Gitter aus Zellen, die jeweils entweder **aktiv** (1) oder **inaktiv** (0) sind. In jedem Schritt schaut jede Zelle auf ihre Nachbarn und aktualisiert ihren Zustand nach einer festen Regel. Viele Schritte hintereinander erzeugen überraschend komplexe, oft musikalische Muster aus sehr einfachen Ausgangsbedingungen.

Wir verwenden einen **Wolfram-Elementarautomaten**: eine Zeile von Zellen, von denen jede auf ihren linken Nachbarn, sich selbst und ihren rechten Nachbarn schaut — drei Zellen, acht mögliche Kombinationen. Die Regelnummer (0–255) legt fest, was in jedem Fall passiert.

12.2 Gesamtüberblick

Der vollständige Datenfluss:



Der zelluläre Automat erzeugt **sowohl** die Melodie als auch den Rhythmus aus einer einzigen Quelle — nur auf zwei verschiedene Arten gelesen. Genau das macht die Technik musikalisch kohärent: Tonhöhe und Rhythmus teilen dieselbe zugrundeliegende Struktur.

12.3 Schritt 1 — Importe

```
from GreasyPidgin.TimeGrid import TimeGrid, beatToSec
from GreasyPidgin.Normalisation import scale
from GreasyPidgin import spntom
from GreasyPidgin.PitchGrid import PitchGrid
from GreasyPidgin.Gesture import Gesture
from GreasyPidgin.CellularAutomata import CellularAutomaton, CellularRules,
intToBinary, binaryToInt
```

Jeder Import stellt ein Werkzeug bereit:

Import	Funktion
TimeGrid	Eine Menge von Zeitpositionen in Sekunden

Import	Funktion
beatToSec	Wandelt eine Taktposition in Sekunden bei gegebenem BPM um
scale	Bildet einen Wert von einem Wertebereich auf einen anderen ab
spntom	Wandelt einen Notennamen wie 'e2' in eine MIDI-Nummer um
PitchGrid	Eine Menge von Tonhöhen in einem gewählten Stimmungssystem und einer Tonleiter
Gesture	Eine musikalische Geste — eine Folge von Phononen (Noten)
CellularAutomaton	Führt den ZA vorwärts in der Zeit aus
CellularRules	Definiert die Regeltabelle des ZA
intToBinary	Wandelt eine ganze Zahl in eine Liste aus 0en und 1en um

12.4 Schritt 2 — Grundlegende Parameter

erstelle im Vorfeld einen Projektordner, in dem du Midi-Dateien speichern kannst

```
path = "MIDI/"
lowestPitchMidi = spntom('e2') # E2 = MIDI 40
highestPitchMidi = spntom('e6') # E6 = MIDI 88
bpm = 161
stepSizeBeats = 0.5 # jede ZA-Zelle = eine Achtelnote
```

spntom (Scientific Pitch Notation to MIDI) wandelt Notennamen in MIDI-Nummern um. 'e2' ist die tiefste Note und 'e6' die höchste — sie legen den Tonhöhenbereich der Melodie fest.

stepSizeBeats = 0.5 bedeutet, dass jede Zelle im ZA-Gitter einem halben Taktschlag entspricht (eine Achtelnote beim gegebenen Tempo).

12.5 Schritt 3 — Das Tonhöhengitter aufbauen

```
pitchGrid = PitchGrid.fromMask(
    'edo12', 'lydianSharp2',
    lowestPitchMidi, highestPitchMidi,
    lowestPitchMidi, lowestPitchMidi
)
```

Ein PitchGrid ist eine Menge erlaubter MIDI-Tonhöhen — eine Art musikalisches Sieb. Anstatt jeden Halbton zuzulassen, beschränken wir die Tonhöhen auf eine bestimmte **Tonleiter innerhalb eines Stimmungssystems**.

- 'edo12' — gleichschwebende Stimmung mit 12 Tönen pro Oktave
- 'lydianSharp2' — ein Modus aus der harmonisch-Moll-Familie: wie Lydisch, aber mit erhöhter zweiter Stufe.
- Der Register reicht von lowestPitchMidi bis highestPitchMidi

Wenn der ZA später "rohe" Midi-Werte erzeugt, rasten wir sie auf dieses Gitter ein — sodass jede Tonhöhe im Ergebnis ein gültiger Skalenton ist.

Note

PitchGrid.fromMask erwartet: Stimmungssystem, Maskenname, tiefste Tonhöhe, höchste Tonhöhe, Stimmungssystem-Grundton, Skalengrundton. Durch Übergabe von lowestPitchMidi für beide Grundtöne wird die Skala auf E verankert.

Binär und Ganzzahl sind zwei Sichtweisen auf dieselbe Zahl.

Jede Zellenzeile im ZA ist heimlich eine ganze Zahl. Die Zellen von links nach rechts als Binärzahl gelesen — 0 0 1 1 0 0 1 0 — ergibt 50. Umgekehrt liefert `intToBinary(50, 8)` wieder [0, 0, 1, 1, 0, 0, 1, 0].

Deshalb hängen der Startzustand `gen0Int = 50` und die Tonhöhenkurve aus `generationsToEnvelope()` zusammen: die Envelope liest jede Generationszeile als ganze Zahl, und diese Zahlen bilden die Melodie. Die Binärliste steuert den Rhythmus. Dieselben Daten, zwei Lesarten.

12.6 Schritt 4 — Den Zellulären Automaten einrichten

```
numGenerations = 40
gen0Int         = 50
gen0            = intToBinary(gen0Int, 16)
```

`gen0` ist der Ausgangszustand — eine Zeile von 16 Zellen. Wir beginnen mit der ganzen Zahl 50, die in Binärdarstellung 0000000000110010 ergibt. Das liefert uns ein bestimmtes Startmuster statt eines zufälligen — die Verwendung einer ganzen Zahl macht es einfach, das Ergebnis zu reproduzieren und zu dokumentieren.

```
ruleNumber = 121
filename    = f"wolfram_{ruleNumber}_gen0_{gen0Int}_\
_numGenerations{numGenerations}.mid"
```

```
wolfram = CellularRules.elementaryWolfram(ruleNumber)
ca       = CellularAutomaton(rules=wolfram, gen0=gen0)
ca.iterate(numGenerations)
```

`CellularRules.elementaryWolfram(140)` erstellt die Nachschlagetabelle für Wolfram-Regel 140. `ca.iterate(40)` führt den Automaten 40 Generationen vorwärts und speichert jede Generation in `ca.generations`.

Der Dateiname kodiert die Regelnummer und die Startbedingungen — unverzichtbar für die Reproduzierbarkeit.

```
env          = ca.generationsToEnvelope()
binaryList   = ca.concatenateGenerations()
```

Den Automaten in der Konsole beobachten

Während der Entwicklung ist es hilfreich, die einzelnen Generationen direkt auszugeben. Jede Generation ist eine Liste von 0en und 1en — die Binärdarstellung einer ganzen Zahl. Diese sind intern in dem dictionary namens `.generations` abgespeichert. Mit `binaryToInt` lässt sich der Zustand jeder Zeile auf einen einzigen Zahlenwert reduzieren, der den "Charakter" dieser Generation zusammenfasst:

```
for key in ca.generations:
    gen      = ca.generations[key]
    intVal   = binaryToInt(gen)
    binStr   = "".join(str(b) for b in gen)  # lesbare Binärdarstellung
    print(f"Generation {key:>3} | {intVal:>6} | {binStr}")
```

```
# Eine typische Ausgabe könnte so aussehen:
# Generation  0 |    50 | 0000000000110010
# Generation  1 |  65465 | 111111110111001
# Generation  2 |   237 | 0000000011101101
```

Die mittlere Spalte — der Ganzzahlwert — ist genau das, was `generationsToEnvelope()` intern berechnet und als Kurve über die Zeit abbildet. Du kannst hier bereits erkennen, ob der Automat in Richtung größerer oder kleinerer Zahlen driftet — und damit, ob die Melodie eher nach oben oder unten tendiert.

Zwei verschiedene Sichtweisen auf die ZA-Ausgabe:

- `generationsToEnvelope()` – wandelt die binären Zellen jeder Generation in eine einzelne ganze Zahl um (indem die Zeile als Binärzahl gelesen wird) und verpackt diese Zahlen als Envelope. Das ergibt eine glatte Kurve, die wir für die Tonhöhenzuordnung verwenden können.
- `concatenateGenerations()` – flacht alle 40 Generationen in eine lange Liste aus 0en und 1en ab ($40 \times 16 = 640$ Werte). Das sind die rohen Rhythmusdaten.

12.7 Schritt 5 – Den ZA auf Tonhöhen abbilden

```
pitchSeq = []
for i in range(numGenerations):
    y = env.getValue(i / numGenerations, True, True)
    rawMidi = scale(y, 0, 1, lowestPitchMidi, highestPitchMidi)
    quantisedMidi = pitchGrid.quantise(rawMidi)
    pitchSeq.append(quantisedMidi)
```

Für jede der 40 Generationen:

1. **Envelope ablesen** an der normierten Zeitposition $i / \text{numGenerations}$ (0.0 arrow.r 1.0). Die `True, True`-Argumente aktivieren die Normierung, sodass der Ausgabewert in $[0, 1]$ liegt.
2. **Skalieren** dieses 0–1-Wertes in den MIDI-Tonhöhenbereich mit `scale()`.
3. **Quantisieren** auf die nächstliegende erlaubte Tonhöhe im `pitchGrid` – Einrasten auf die LydianSharp2-Skala.

Das Ergebnis ist eine Liste von 40 MIDI-Tonhöhen, eine pro Generation, die dem sich entwickelnden Ganzzahlverlauf des ZA folgt.

12.8 Schritt 6 – Den ZA auf Rhythmen abbilden

```
startTimesSec = []
for i, bin in enumerate(binaryList):
    startTimeBeats = i * stepSizeBeats
    if bin == 1:
        startTimeSec = beatToSec(startTimeBeats, bpm)
        startTimesSec.append(startTimeSec)
```

`binaryList` hat 640 Werte (0 oder 1). Jede Position entspricht einem halben Taktschlag. Wenn eine Zelle **1** ist, wird dort ein Notenanschlag gesetzt; bei **0** herrscht Stille. `beatToSec` wandelt die Taktposition mit dem BPM in Sekunden um.

Das Ergebnis ist eine Liste von Einsatzzeiten in Sekunden – nur die Positionen, an denen der ZA eine 1 hat.

```
tg = TimeGrid(startTimesSec, bpm=161)
deltaTs = tg.differentiate()
```

`TimeGrid` verpackt die Einsatzzeiten in eine musikalische Zeitstruktur. `differentiate()` wandelt die absoluten Zeiten in **Intereinsatzintervalle** um (die Zeit zwischen aufeinanderfolgenden Einsätzen) – das Format, das `Gesture.fromLists` benötigt.

Note

`differentiate()` ist an die gleichnamige SuperCollider-Methode angelehnt, wobei in dieser Version standardmäßig der erste Wert als 0 angenommen wird - nicht als Differenz von 0.

12.9 Schritt 7 — Die Geste aufbauen

```
g = Gesture.fromLists(  
    numGenerations, 0,  
    deltaTs, 0.125, pitchSeq, 100,  
    bpm=bpm  
)  
g.toMidi(path + filename)
```

`Gesture.fromLists` fügt alles zu einer musikalischen Geste zusammen:

Argument	Wert	Bedeutung
size	numGenerations (40)	Anzahl der Noten
startOffset	0	Beginn bei Zeit null
startDeltas	deltaTs	Intereinsatzintervalle in Sekunden
durations	0.125	Feste Notendauer (Achtelnote)
pitches	pitchSeq	Die 40 MIDI-Tonhöhen aus dem ZA
dynamics	100	Feste Anschlagstärke
bpm	161	Tempo für den MIDI-Export

Schließlich schreibt `g.toMidi(...)` die MIDI-Datei. Der Dateiname kodiert die Regelnummer und die Anfangsbedingungen, sodass du exakt dasselbe Ergebnis jederzeit neu erzeugen kannst.

12.10 Experimente

Regelnummer ändern:

```
ruleNumber = 30 # chaotisch  
ruleNumber = 110 # komplex, Turing-vollständig  
ruleNumber = 90 # Sierpinski-Dreieck
```

Startmuster ändern:

```
gen0Int = 1 # einzelne aktive Zelle in der Mitte  
gen0Int = 255 # alle Zellen aktiv  
gen0Int = 42 # beliebig – dokumentieren und anhören
```

Tonleiter ändern:

```
pitchGrid = PitchGrid.fromMask('edo12', 'phrygian', ...)  
pitchGrid = PitchGrid.fromMask('werckmeister1', 'dorian', ...)
```

Schrittgröße ändern:

```
stepSizeBeats = 0.25 # Sechzehntelnoten – dichtere Rhythmen  
stepSizeBeats = 1.0 # Viertelnoten – spärlicher
```

Jede Kombination aus Regel, Startmuster und Tonleiter ergibt eine einzigartige Komposition. Die Dateinamen-Konvention (`wolfram_140_gen0_50_numGenerations40.mid`) macht es leicht, deine Erkundungen zu katalogisieren.

12.11 Ausblick: Die Stärke algorithmischer Generierung

Einer der größten Vorteile des algorithmischen Komponierens ist die Möglichkeit, **viele Varianten aus denselben Parametern** zu erzeugen — mit minimalem Aufwand. Statt eine Regel manuell zu wählen, können wir einfach einen ganzen Bereich von Regeln durchlaufen und für jede eine eigene MIDI-Datei erstellen.

Das folgende Beispiel zeigt außerdem eine weitere Möglichkeit, den Startzustand zu definieren: statt einer ganzen Zahl schreiben wir das Bitmuster **direkt als Liste** und

berechnen den Ganzzahlwert nachträglich mit `binaryToInt`. Das macht die rhythmische Struktur des Startmusters unmittelbar lesbar:

```
gen0 = [  
    1,0,0,1,0,0,1,0,  
    0,1,1,0,1,1,0,1]  
  
gen0Int = binaryToInt(gen0)
```

Hier sieht man auf einen Blick: die erste Hälfte hat ein punktiertes Muster (1 0 0 1 0 0 1 0), die zweite eine dichtere Gegenbewegung (0 1 1 0 1 1 0 1). Diese Struktur wird als Ausgangszustand in den Automaten eingespeist – und jede Regel entwickelt sie auf ihre eigene Weise weiter.

Anstatt nun eine einzelne Regel auszuprobieren, iterieren wir über **50 aufeinanderfolgende Regeln** (100–149) und schreiben für jede eine eigene Datei:

```
from GreasyPidgin.CellularAutomata import binaryToInt  
from GreasyPidgin.TimeGrid import TimeGrid, beatToSec  
from GreasyPidgin.Normalisation import scale  
from GreasyPidgin import spntom  
from GreasyPidgin.PitchGrid import PitchGrid  
from GreasyPidgin.Gesture import Gesture  
from GreasyPidgin.CellularAutomata import CellularAutomaton, CellularRules,  
intToBinary  
  
path = "MIDI/"  
  
lowestPitchMidi = spntom('e2')  
highestPitchMidi = spntom('e6')  
bpm = 161  
stepSizeBeats = 0.5  
  
pitchGrid = PitchGrid.fromMask(  
    'edol2', 'lydianSharp2',  
    lowestPitchMidi, highestPitchMidi, lowestPitchMidi, lowestPitchMidi)  
  
numGenerations = 40  
  
gen0 = [1,0,0,1,0,0,1,0,  
        0,1,1,0,1,1,0,1]  
gen0Int = binaryToInt(gen0)  
  
for ruleNumber in range(100, 150):  
    filename = f"wolfram_{ruleNumber}_gen0_{gen0Int}"  
    _numGenerations{numGenerations}.mid"  
    wolfram = CellularRules.elementaryWolfram(ruleNumber)  
    ca = CellularAutomaton(rules=wolfram, gen0=gen0)  
    ca.iterate(numGenerations)  
    env = ca.generationsToEnvelope()  
    binaryList = ca.concatenateGenerations()  
  
    pitchSeq = []  
    for i in range(numGenerations):  
        y = env.getValue(i / numGenerations, True, True)  
        rawMidi = scale(y, 0, 1, lowestPitchMidi, highestPitchMidi)  
        quantisedMidi = pitchGrid.quantise(rawMidi)  
        pitchSeq.append(quantisedMidi)  
  
    startTimesSec = []  
    for i, bin in enumerate(binaryList):  
        startTimeBeats = i * stepSizeBeats  
        if bin == 1:  
            startTimeSec = beatToSec(startTimeBeats, bpm)  
            startTimesSec.append(startTimeSec)
```

```
tg      = TimeGrid(startTimesSec, bpm=161)
deltaTs = tg.differentiate()

g = Gesture.fromLists(
    numGenerations, 0,
    deltaTs, 0.125, pitchSeq, 100, bpm=bpm
)
g.toMidi(path + filename)
```

Mit einem einzigen Skript entstehen so **50 MIDI-Dateien** — jede klingt anders, alle teilen dieselbe rhythmische DNA des Startmusters und dieselbe Tonleiter. In einer DAW kannst du diese Dateien nebeneinander laden und gezielt auswählen, welche Varianten sich für dein Stück eignen.

Kompositorischer Workflow

Algorithmische Generierung ersetzt nicht die kompositorische Entscheidung — sie verlagert sie. Statt jede Note einzeln zu setzen, wählst du:

1. **Das Startmuster** (gen0) — die rhythmische DNA
2. **Die Regeln** — welche Automaten du als Kandidaten zulässt
3. **Die Tonleiter** — den harmonischen Raum
4. **Die Auswahl** — welche der generierten Varianten du verwendest

Das Stück entsteht im Dialog zwischen Algorithmus und Gehör.

13. Markov-Melodie

Eine Einführung in die Erzeugung von Melodien mit Markov-Ketten und GreasyPidgin.

Am Ende dieses Tutorials hast du eine MIDI-Datei mit einer Melodie, deren **Tonhöhen** durch ein Markov-Modell bestimmt werden — ein Modell, das lernt, welche Töne in einer musikalischen Folge aufeinander folgen.

13.1 Was ist eine Markov-Kette?

Eine Markov-Kette ist ein stochastisches Modell, das beschreibt, wie ein System von einem Zustand in den nächsten übergeht. Die zentrale Eigenschaft: der nächste Zustand hängt **nur vom aktuellen Zustand** ab — nicht von der gesamten Vorgeschichte.

In der Musik bedeutet das: gegeben der aktuelle Ton, mit welcher Wahrscheinlichkeit folgt welcher nächste Ton? Diese Wahrscheinlichkeiten werden in **Übergangsregeln** kodiert.

13.2 Gesamtüberblick

Der vollständige Datenfluss:

```
Markov-Regeln (Tonhöhen + Gewichte)
↓
MarkovChain.getNElements()
↓
Liste von MIDI-Tonhöhen
↓
Gesture.fromLists()
↓
MIDI-Datei
```

13.3 Schritt 1 — Importe

```
from GreasyPidgin.Gesture import Gesture
from GreasyPidgin.MarkovChains import MarkovChain, MarkovRule, MarkovRules
```

Import	Funktion
Gesture	Eine musikalische Geste — eine Folge von Noten
MarkovChain	Erzeugt Sequenzen durch Markov-Übergänge
MarkovRule	Eine einzelne Übergangsregel: von einem Zustand zu möglichen Nachfolgern
MarkovRules	Eine validierte Sammlung von MarkovRule-Objekten

13.4 Schritt 2 — Übergangsregeln definieren

```
rule60 = MarkovRule(60, [63, 64, 67, 68], [4,1,4,4])
rule63 = MarkovRule(63, [60,67,68],[1,2,1])
rule64 = MarkovRule(64, [60],[1])
rule67 = MarkovRule(67, [60,63,68],[1,1,2])
rule68 = MarkovRule(68, [60,67],[1,2])
```

Jede MarkovRule hat drei Argumente:

Argument	Bedeutung
Erstes	Der aktuelle Zustand (MIDI-Tonhöhe)
Zweites	Liste möglicher Nachfolgetöne
Drittes	Relative Gewichte (höher = wahrscheinlicher)

rule60 bedeutet: wenn der aktuelle Ton MIDI 60 (C4) ist, folgt mit hoher Wahrscheinlichkeit MIDI 63 oder 68, seltener 64.

Note

Die Gewichte werden automatisch normiert — es kommt nur auf ihre Verhältnisse zueinander an, nicht auf absolute Werte. [4, 1, 4, 4] bedeutet, dass 63, 67 und 68 je viermal so wahrscheinlich sind wie 64.

Die hier definierten Töne — 60, 63, 64, 67, 68 — entsprechen c4, ds4, e4, g4, Ab4

13.5 Schritt 3 — Die Markov-Kette aufbauen

```
seed = 60
numElements = 200
```

```
mr = MarkovRules([rule60, rule63, rule64, rule67, rule68])
mc = MarkovChain(rules=mr, seed=seed)
```

MarkovRules überprüft die Vollständigkeit der Regeln: jeder Ton, der als Nachfolger auftauchen kann, muss auch selbst eine Regel haben. Fehlt eine Regel, gibt es eine Warnung.

seed = 60 setzt den Startzustand — die erste Tonhöhe der Sequenz.

13.6 Schritt 4 — Sequenz generieren

```
pitchListMidi = mc.getNElements(n=numElements)
```

getNElements(n=200) erzeugt 200 Töne durch sukzessive Anwendung der Übergangsregeln. Das Ergebnis ist eine Liste von MIDI-Tonhöhen — die Melodie.

Tip

getNElements gibt auch den Startzustand zurück. Die Liste hat also $n + 1 = 201$ Elemente: [seed, t1, t2, ..., t200]. Das size-Argument von Gesture.fromLists sollte daher ebenfalls numElements + 1 oder numElements sein, je nachdem ob du den Seed mithörst oder nicht.

13.7 Schritt 5 — Geste aufbauen und exportieren

```
Gesture.fromLists(numElements, 0, 0.5, 0.5, pitchListMidi, bpm=161,
timeUnit='beats').toMidi()
```

Argument	Wert	Bedeutung
size	numElements (200)	Anzahl der Noten
startOffset	0	Beginn bei Taktschlag 0
startDeltas	0.5	Jede Note beginnt einen halben Beat nach der vorherigen
durations	0.5	Jede Note dauert einen halben Beat
pitches	pitchListMidi	Die 200 Markov-Tonhöhen
bpm	161	Tempo
timeUnit	'beats'	Zeiten in Taktschlägen angegeben

13.8 Experimente

Andere Starttöne:

```
seed = 63 # beginnt auf D#4  
seed = 67 # beginnt auf G4
```

Andere Gewichte — mehr Wiederholungen:

```
rule60 = MarkovRule(60, [60, 63, 67, 68], [8, 2, 2, 2]) # bleibt oft auf C4
```

Längere Sequenz:

```
numElements = 500
```

Andere Notendauern — 16tel statt 8tel:

```
Gesture.fromLists(numElements, 0, 0.25, 0.25, pitchListMidi, bpm=161,  
timeUnit='beats').toMidi()
```


14. Markov-Rhythmus

Eine Einführung in die Erzeugung von Rhythmen mit Markov-Ketten und GreasyPidgin.

Am Ende dieses Tutorials hast du eine MIDI-Datei mit einem Rhythmusmuster, dessen **Notenwerte** durch ein Markov-Modell bestimmt werden — ein Modell, das lernt, wie Notenwerte in einer musikalischen Folge aufeinander folgen.

14.1 Rhythmus als Markov-Kette

Rhythmus lässt sich als eine Folge von Notenwerten beschreiben: Viertel, Achtel, Sechzehntel, Halbe. Eine Markov-Kette kann lernen, welche Notenwerte in einer musikalischen Folge wahrscheinlich aufeinander folgen — zum Beispiel, dass auf eine Sechzehnte meist weitere Sechzehntel folgen, aber gelegentlich eine Halbe als Variation kommt.

14.2 Gesamtüberblick

```
Markov-Regeln (Notenwerte + Gewichte)
↓
MarkovChain.getNElements()
↓
Liste von Notenwerten (als Beat-Brüche)
↓
Skalierung auf Taktschläge (* 4)
↓
Gesture.fromLists()
↓
MIDI-Datei
```

14.3 Schritt 1 — Importe

```
from GreasyPidgin.Gesture import Gesture
from random import choices
from GreasyPidgin.MarkovChains import MarkovChain, MarkovRule, MarkovRules
```

choices aus Pythons random-Modul wird hier für die zufällige Auswahl der Tonhöhen verwendet — ohne Markov-Logik, da der Fokus auf dem Rhythmus liegt.

14.4 Schritt 2 — Rhythmische Übergangsregeln definieren

```
ruleHalbe = MarkovRule(1/2, [1/2, 1/4, 1/16], [1, 1, 0.5])
rule4tel = MarkovRule(1/4, [1/2, 1/8], [1, 1])
rule8tel = MarkovRule(1/8, [1/16, 1/8], [1, 4])
rule16tel = MarkovRule(1/16, [1/16, 1/2], [8, 1])
```

Die Zustände sind hier keine Tonhöhen, sondern **Notenwerte als Brüche**:

Zustand	Notenwert
1/2	Halbe Note
1/4	Viertelnote
1/8	Achtelnote
1/16	Sechzehntelnote

Die Gewichte beschreiben musikalisches Verhalten:

- rule8tel: nach einer Achtelnote folgt viermal so häufig wieder eine Achtelnote wie eine Sechzehntelnote — **Läufe bleiben Läufe**.
- rule16tel: nach einer Sechzehntel folgt achtmal so häufig wieder eine Sechzehntel wie eine Halbe — **dichte Stellen bleiben dicht**, aber gelegentliche Pausen kommen vor.

- ruleHalbe: nach einer Halben ist eine weitere Halbe oder eine Viertel gleich wahrscheinlich – **Ruhepunkte führen zu weiteren Ruhepunkten.**

Note

Die Notenwerte werden als Brüche einer ganzen Note angegeben. 1/4 bedeutet also nicht „eine Viertelsekunde“, sondern eines ‘Viertelnote’, also **einem Schlag**. Die spätere Skalierung (* 4) wandelt diese in Taktschläge -beats- um.

14.5 Schritt 3 – Sequenz generieren

seed = 1/2

numElements = 200

```
mr = MarkovRules([ruleHalbe, rule4tel, rule8tel, rule16tel])
```

```
mc = MarkovChain(rules=mr, seed=seed)
```

```
rhthmsRaw = mc.getNElements(n=numElements)
```

Die Kette beginnt mit einer Halben (seed = 1/2) und erzeugt 200 Notenwerte nach den definierten Übergangswahrscheinlichkeiten.

14.6 Schritt 4 – Skalierung auf Taktschläge

```
rhthmsMatched = [r * 4 for r in rhthmsRaw]
```

Gesture.fromLists mit timeUnit='beats' erwartet Zeiten in Taktschlägen. Da unsere Notenwerte als **Brüche einer ganzen Note** definiert sind (ganze Note = 4 beats), multiplizieren wir mit 4:

Rohwert	× 4	Bedeutung
1/2	2.0	2 beats = Halbe
1/4	1.0	1 beats = Viertelnote
1/8	0.5	0.5 beats = Achtelnote
1/16	0.25	0.25 beats = Sechzehntelnote

14.7 Schritt 5 – Tonhöhen zufällig auswählen

```
pitchListMidi = choices([48, 55, 60, 61], [4, 2, 4, 1], k=numElements)
```

Da hier der Rhythmus im Mittelpunkt steht, werden die Tonhöhen einfach zufällig aus einer kleinen Auswahl gezogen – ohne jede Markov-Logik. choices aus Pythons random-Modul funktioniert wie ein gewichtetes Lotto:

```
pitchListMidi = choices([48, 55, 60, 61], [4, 2, 4, 1], k=numElements)
```

Argument	Bedeutung
[48, 55, 60, 61]	Die möglichen Tonhöhen: C3, G3, C4, Df4
[4, 2, 4, 1]	Relative Gewichte: C3 und C4 viermal so häufig wie Df4
k=numElements	Anzahl der zu ziehenden Werte (200)

Im Gegensatz zu einer Markov-Kette hat jede Ziehung **kein Gedächtnis** – die Wahrscheinlichkeiten ändern sich nie in Abhängigkeit vom vorherigen Ton.

Note

choices wählt **mit Zurücklegen** — dieselbe Tonhöhe kann beliebig oft hintereinander gezogen werden.

14.8 Schritt 6 — Geste aufbauen und exportieren

```
Gesture.fromLists(  
  numElements, 0,  
  rhtmsMatched, rhtmsMatched,  
  pitchListMidi, bpm=161,  
  timeUnit='beats').toMidi()
```

Hier werden sowohl startDeltas als auch durations auf dieselbe Liste rhtmsMatched gesetzt — jede Note beginnt genau dann, wenn die vorherige endet (**legato**, kein Überlappen, keine Pause).

14.9 Experimente

Mehr Sechzehntel-Läufe:

```
rule16tel = MarkovRule(1/16, [1/16, 1/2], [16, 1]) # noch seltener  
unterbrochen
```

Andere Startsituation:

```
seed = 1/16 # beginnt mit einem Sechzehntel-Lauf  
seed = 1/4  # beginnt gemächlich
```

Pausen einbauen — indem du einen Zustand 0 (Pause) einführest:

```
rulePause = MarkovRule(0, [1/4, 1/8], [1, 1])  
ruleHalbe = MarkovRule(1/2, [1/2, 1/4, 0], [1, 1, 1]) # Halbe kann in Pause  
übergehen
```


15. Markov-Intervalle

Eine Einführung in die Erzeugung von Melodien über **Intervall-Markov-Ketten** mit GreasyPidgin.

Im Unterschied zur direkten Tonhöhen-Markov-Kette werden hier nicht die absoluten Tonhöhen als Zustände verwendet, sondern **Intervalle** — die Abstände zwischen aufeinanderfolgenden Tönen in Halbtönen. Das Ergebnis ist eine Melodie, deren Bewegungscharakter (Sprünge, Schritte, Wiederholungen) durch das Modell gelernt wurde — unabhängig von der Ausgangstonhöhe.

15.1 Intervalle statt Tonhöhen

Der entscheidende Unterschied zur Melodie-Variante:

Ansatz	Zustand	Modelliert
Tonhöhen-Markov	MIDI-Nummer (60, 63, ...)	Welcher Ton folgt
Intervall-Markov	Halbton-Abstand (-7, -3, 0, ...)	Welche Bewegung folgt

Die Intervall-Variante ist **transponierfähig**: dasselbe Modell klingt in jeder Tonart gleich, weil es die melodische Geste beschreibt, nicht die absoluten Töne.

15.2 Gesamtüberblick

```
Markov-Regeln (Intervalle + Gewichte)
↓
MarkovChain.getNElements()
↓
Liste von Intervallen (Halbtöne)
↓
Integration: Startnote + kumulative Summe der Intervalle
↓
forceIntoRangeMidi: Oktavkorrektur
↓
Gesture.fromLists()
↓
MIDI-Datei
```

15.3 Schritt 1 — Importe

```
from GreasyPidgin.Pitch import forceIntoRangeMidi
from GreasyPidgin.Gesture import Gesture
from GreasyPidgin.MarkovChains import MarkovChain, MarkovRule, MarkovRules
```

`forceIntoRangeMidi` ist neu gegenüber der Melodie-Variante: es verschiebt Töne, die außerhalb des gewünschten Registers liegen, um Oktaven nach oben oder unten — ohne die Tonhöhenklasse zu verändern.

15.4 Schritt 2 — Intervall-Regeln definieren

```
rule5Down = MarkovRule(-7, [0, 5, 1], [2, 1, 1])
ruleM3Down = MarkovRule(-3, [0, 5, -7], [4, 2, 1])
ruleM2Down = MarkovRule(-1, [1, -7], [1, 1])
rule1 = MarkovRule(0, [0, -1, 1, 5, -7], [1, 2, 2, 2, 1])
ruleM2Up = MarkovRule(1, [0, -1, 3], [2, 1, 1])
ruleM3Up = MarkovRule(3, [1, -3, 3], [4, 1, 4])
rule4Up = MarkovRule(5, [-1, 1], [1, 1])
```

Die Zustände sind Intervalle in Halbtönen:

Zustand	Intervall
---------	-----------

Zustand	Intervall
-7	Quinte abwärts
-3	Kleine Terz abwärts
-1	Kleiner Sekundschritt abwärts
0	Wiederholung (kein Sprung)
1	Kleiner Sekundschritt aufwärts
3	Kleine Terz aufwärts
5	Quarte aufwärts

Note

Das Modell beschreibt eine Melodik, die sich bevorzugt in kleinen Schritten bewegt, gelegentlich Terzen springt und selten große Sprünge macht.

Die Übergangslogik beschreibt musikalisches Verhalten:

- Nach einer **Wiederholung** (0) folgen am häufigsten Sekundschritte in beide Richtungen oder eine **Quarte aufwärts** – die Melodie „entscheidet sich“.
- Nach einer **kleinen Terz aufwärts** (3) folgt meist wieder eine kleine Terz oder ein Sekundschritt aufwärts – **sequenzartige Bewegungen** entstehen.
- Nach einer **Quinte abwärts** (-7) folgt oft eine Wiederholung – der Sprung landet auf einer wichtigen Note und bleibt dort kurz.

15.5 Schritt 3 – Intervallsequenz generieren

seed = 1

numElements = 200

```
mr = MarkovRules([rule1, rule5Down, ruleM3Down, ruleM2Down, ruleM2Up,
ruleM3Up, rule4Up])
mc = MarkovChain(rules=mr, seed=seed)
```

```
intervalListMidi = mc.getNElements(n=numElements)
```

seed = 1 bedeutet: die Kette beginnt mit einem Sekundschritt aufwärts. Die Ausgabe ist eine Liste von 200 Intervallen.

15.6 Schritt 4 – Intervalle in Tonhöhen umrechnen

```
pitchListMidi = [57]
for intrvl in intervalListMidi:
    candidate = pitchListMidi[-1] + intrvl
    pitchMidi = forceIntoRangeMidi(candidate, 48, 90)
    pitchListMidi.append(pitchMidi)
```

Der Startton ist MIDI 57 (A3). Jedes Intervall wird auf den letzten Ton addiert – das ist **Integration**: aus Differenzen werden absolute Werte.

forceIntoRangeMidi(candidate, 48, 90) stellt sicher, dass die Melodie im Register A2–Fs6 bleibt. Töne, die diesen Bereich nach oben oder unten verlassen würden, werden um eine Oktave verschoben.

Integration – von Intervallen zu Tonhöhen

Das Intervall-Modell lernt die **Bewegung** einer Melodie. Die Integration rekonstruiert daraus die absoluten Töne – beginnend von jedem beliebigen Startton.

15.7 Schritt 5 — Geste aufbauen und exportieren

```
Gesture.fromLists(numElements, 0, 0.5, 0.5, pitchListMidi, bpm=161,  
timeUnit='beats').toMidi()
```

Identisch zur Melodie-Variante — der einzige Unterschied liegt in der Herkunft von pitchListMidi.

15.8 Experimente

Anderen Startnoten:

```
pitchListMidi = [60]    # C4  
pitchListMidi = [69]    # A4
```

Engeres Register — mehr Oktavkorrekturen:

```
pitchMidi = forceIntoRangeMidi(candidate, 55, 79)    # G3 bis G5
```

Andere Sprunggewichtung — lyrischer:

```
ruleM3Up = MarkovRule(3, [1, 3], [3, 1])    # Terz führt meist zu Sekundschrift  
rule5Down = MarkovRule(-7, [1, 0], [2, 1])    # Quintfall führt aufwärts
```

Sequenzielle Muster verstärken:

```
ruleM3Up = MarkovRule(3, [1, -3, 3], [1, 1, 8])    # Terz-Sequenzen dominieren
```


16. Markov-Melodie mit Markov-Rhythmus

Eine Einführung in die gleichzeitige Verwendung von zwei unabhängigen Markov-Ketten: eine für **Tonhöhen** (über Intervalle), eine für **Rhythmus** (über Notenwerte).

Dies ist die Zusammenführung der beiden vorherigen Tutorials. Zwei unabhängige stochastische Prozesse erzeugen je eine eigene Dimension der Musik — und werden am Ende in einer einzigen Geste kombiniert.

16.1 Zwei Ketten, eine Musik

Das Entscheidende an diesem Ansatz: Tonhöhe und Rhythmus sind **entkoppelt**. Jede Dimension hat ihre eigene Markov-Logik, ihre eigene Entwicklung. Beide sind durch Übergangswahrscheinlichkeiten gestaltet — aber sie kennen einander nicht.

Das erzeugt eine charakteristische Textur: rhythmische Muster und melodische Gesten entwickeln sich unabhängig voneinander, überlagern sich zufällig und erzeugen dabei eine Komplexität, die mit rein manueller Komposition schwer zu erreichen wäre.

16.2 Gesamtüberblick



16.3 Schritt 1 — Importe

```
from GreasyPidgin.Pitch import forceIntoRangeMidi
from GreasyPidgin.Gesture import Gesture
from random import choices
from GreasyPidgin.MarkovChains import MarkovChain, MarkovRule, MarkovRules
```

16.4 Schritt 2 — Intervall-Kette (Tonhöhen)

```
rule5Down = MarkovRule(-7, [0, 5, 1], [2, 1, 1])
ruleM3Down = MarkovRule(-3, [0, 5, -7], [4, 2, 1])
ruleM2Down = MarkovRule(-1, [1, -7], [1, 1])
rule1 = MarkovRule(0, [0, -1, 1, 5, -7], [1, 2, 2, 2, 1])
ruleM2Up = MarkovRule(1, [0, -1, 3], [2, 1, 1])
ruleM3Up = MarkovRule(3, [1, -3, 3], [4, 1, 4])
rule4Up = MarkovRule(5, [-1, 1], [1, 1])

seed = 1
numElements = 200

mr = MarkovRules([rule1, rule5Down, ruleM3Down, ruleM2Down, ruleM2Up,
ruleM3Up, rule4Up])
mc = MarkovChain(rules=mr, seed=seed)

intervallListMidi = mc.getNElements(n=numElements)

pitchListMidi = [57]
for intrvl in intervallListMidi:
    candidate = pitchListMidi[-1] + intrvl
```



```
pitchMidi = forceIntoRangeMidi(candidate, 48, 90)
pitchListMidi.append(pitchMidi)
```

Identisch mit dem Intervall-Tutorial. Die Kette erzeugt 200 Intervalle, die dann zu absoluten Tonhöhen integriert werden. Startnote: A3 (MIDI 57), Register: A2–F#6.

16.5 Schritt 3 – Rhythmus-Kette

```
ruleHalbe = MarkovRule(1/2, [1/2, 1/4, 1/16], [1, 1, 0.5])
rule4tel = MarkovRule(1/4, [1/2, 1/8], [1, 1])
rule8tel = MarkovRule(1/8, [1/16, 1/8], [1, 4])
rule16tel = MarkovRule(1/16, [1/16, 1/2], [8, 1])

seedRhythm = 1/2

mrRhythm = MarkovRules([ruleHalbe, rule4tel, rule8tel, rule16tel])
mcRhythm = MarkovChain(rules=mrRhythm, seed=seedRhythm)

rhythmsRaw = mcRhythm.getNElements(n=numElements)
rhythmsMatched = [r * 4 for r in rhythmsRaw]
```

Identisch mit dem Rhythmus-Tutorial. Die Kette erzeugt 200 Notenwerte als Taktbrüche, die dann in Taktschläge skaliert werden.

Note

Beide Ketten erzeugen `numElements = 200` Elemente. Die Listen `pitchListMidi` und `rhythmsMatched` haben also dieselbe Länge – Voraussetzung dafür, dass `Gesture.fromLists` sie paarweise zuordnet.

Der Seed von `pitchListMidi` ergibt technisch 201 Elemente (`[57] + 200` generierte). Das erste Element ist der Startnoten-Seed und wird durch `size = numElements` automatisch auf die ersten 200 Elemente begrenzt.

16.6 Schritt 4 – Beide Ketten kombinieren

```
Gesture.fromLists(numElements, 0, rhythmsMatched, rhythmsMatched, pitchListMidi,
bpm=161, timeUnit='beats').toMidi()
```

Die Kombinationslogik ist einfach: `Gesture.fromLists` nimmt die Listen als parallele Streams entgegen. Element `i` von `pitchListMidi` wird mit Element `i` von `rhythmsMatched` kombiniert – erste Note mit erster Notendauer, zweite Note mit zweiter Notendauer usw.

Die Unabhängigkeit der beiden Ketten bedeutet: ein langer Ton (Halbe) kann genauso gut auf einem Ton landen, der einem langen Sprung folgte, wie auf einem, der einem Sekundschrift folgte. Diese zufällige Überlagerung erzeugt den charakteristischen Klang des Stücks.

Kompositorische Kontrolle durch Gewichtung

Auch wenn die beiden Ketten unabhängig sind, lässt sich ihr Zusammenspiel indirekt steuern:

- Hohe Wiederholungsgewichtung in der Intervall-Kette `arrow.r` statischere Melodie
- `arrow.r` rhythmischeres Muster tritt stärker hervor
- Viele Sechzehntel in der Rhythmus-Kette `arrow.r` schnelle Läufe, bei denen melodische Sprünge besonders auffallen
- Lange Töne (Halbe, Ganze) in der Rhythmus-Kette `arrow.r` einzelne Intervallbewegungen bekommen mehr Gewicht

16.7 Experimente

Rhythmisch dichtere Version:

```
rule16tel = MarkovRule(1/16, [1/16, 1/2], [16, 1]) # Sechzehntel-Läufe dominieren
```

Melodisch ruhigere Version:

```
rule1 = MarkovRule(0, [0, -1, 1, 5, -7], [8, 2, 2, 1, 1]) # mehr Wiederholungen
```

Andere Startsituation:

```
seed      = -3      # beginnt mit kleiner Terz abwärts  
seedRhythm = 1/16   # beginnt mit Sechzehntel-Lauf
```

Ungleiche Elementzahlen — Rhythmus als Schleife über die Melodie:

```
numRhythm = 50  
rhythmsRaw = mcRhythm.getNElements(n=numRhythm)  
rhythmsMatched = [r * 4 for r in rhythmsRaw]  
# fromLists wiederholt rhythmsMatched zyklisch über 200 Noten  
Gesture.fromLists(numElements, 0, rhythmsMatched, rhythmsMatched, pitchListMidi,  
bpm=161, timeUnit='beats').toMidi()
```

Durch unterschiedliche Längen entstehen **Polyrhythmen zwischen Melodie und Rhythmik** — das rhythmische Pattern wiederholt sich alle 50 Noten, die melodische Bewegung alle 200.